Boot

hangs at “booting...”

On some serial connected devices the console settings are different, in which case you would not be able to start the installer.

If you can reach the loader prompt, you could try to change some kernel parameters before booting.

```
set hint.uart.0.flags=0x0
set hint.uart.1.flags=0x10
set hint.uart.0.at="isa"
set comconsole_speed=115200
set comconsole_port=0x3F8
set console=comconsole
boot
```

Note

To enter the loader prompt press `3` when the OPNsense boot menu is visible

After installation, you could persist these settings in **System ▸ Settings ▸ Tunables**

Restore Configuration via Console

Sometimes you may accidentally introduce a breaking change that interrupts network access, prevents user login or makes SSH and WebGUI inaccessible. Rolling back without a reinstall is possible if there is serial and console access.

For serial access to the console, reference this guide: Serial Console connectivity

Console access is also possible directly when running a virtual machine or hardware with VGA capabilities.

This guide describes step by step how to restore a previous configuration via console.

Attention

This requires automatic backups being retained. If you changed *Backup Count* settings in **System ‣ Configuration ‣ Backups**, the history can be too short to restore a working configuration from the local backup cache. Mounting a USB flash drive with a previous configuration might be necessary.

Tip

Using Snapshots with a ZFS filesystem can make rollbacks simpler.

1. Power off the device

Since you need to influence the boot process to reset the configuration, you should start with powering off the (virtual) device.

Attention

If you run a HA setup, ensure any cron jobs that synchronize the configuration are turned off. Otherwise they can overwrite the configuration that you restored.

2. Access console

Make sure you are able to access the (virtual) console, in case it is a physical machine you might want to connect a monitor and keyboard, when it is an appliance with serial access, make sure to connect to the serial/usb port using an application like putty.

Tip

When using devices from the OPNsense shop (https://shop.opnsense.com), usually there's a mini-usb to serial cable included in the box, the Serial Console connectivity guide will help you with the setup.

3. Power on and boot

Switch the power and wait for the OPNsense splash screen to appear. Do not interact with the splash screen, wait for the actual boot to start.

When the text scrolls fast, hold the **CTRL** button and press **C** rapidly to break out of the boot process into a shell. This does not need any authentication.

4. Replace configuration

Now that you are in the shell, you can do changes to the filesystem. In our example, we will restore a previous config.xml version.

First we will evaluate which config.xml version should be restored:

```
cd /conf/backup
ls -la
```

Check out the timestamps of the backup configurations, copy the filename of one that was before you made the breaking change. We will backup our current config.xml and then overwrite it with a previous version.

Attention

The below example must be adjusted to represent your config.xml timestamp.

```
cp /conf/config.xml /conf/config.xml.backup
cp /conf/backup/config-YOURTIMESTAMP.xml /conf/config.xml
```

Reboot and the replaced configuration will be loaded.

```
reboot
```

After the reboot, confirm that you can log in and that the breaking change has been rolled back. If not, repeat the above steps and go back further with the backup configuration timestamp.

If this cannot fix it, reinstall your appliance with the latest available image and restore a known-good configuration you have kept safe.

Gateways and monitoring

The address you are trying to monitor should be reachable using the interface the gateway is attached to, either directly or using a static route (check **System ▸ Routes ▸ Status**).

dpinger:.. sendto error: XXX

Usually found in **System ▸ Log Files ▸ General**, every code has a meaning, usually explained in errno.h (`man errno`)

Some common ones are explained in the Common error codes section.

arpresolve: can't allocate llinfo for..

This message usually means that the configured gateway lies outside the configured subnets for this firewall (for IPv4).

Tip

Double check the subnets of your interface and virtual IP's, you can also use **Interfaces ▸ Overview** for a quick list of all configured addresses.

System hardening vs performance

OPNsense tends to choose more strict hardening options by default, so when comparing performance between upstream standard FreeBSD it's good to know which settings differ and can have an impact on your measurements. This document aims to describe (some of) the differences, so when you value performance over security it is more obvious which toggles might be worthwhile to change.

Keep in mind that most of the settings will need a reboot and can be altered using system tunables in **System ▸ Settings ▸ Tunables**.

IPv4 random ID's [net.inet.ip.random_id]

control IP(v4) IDs generation behaviour. This closes a minor information leak which allows remote observers to determine the rate of packet generation on the machine by watching the counter. At the same time, on high-speed links, it can decrease the ID reuse cycle greatly. IPv6 flow IDs and fragment IDs are always random. (source `man -S 4 inet`)

Our default is 1 (enabled).

Spectre and Meltdown

To mitigate some of the speculative execution vulnerabilities, there are a couple of settings available in FreeBSD. More information about the various vulnerabilities and associated patches can be found here

Meltdown mitigation using Page Table Isolation (PTI), although also enabled in FreeBSD it's worth to mention which setting is responsible for enabling this feature. To disable PTI set `vm.pmap.pti` to 0. Not all cpu's are vulnerable for Meltdown, in which case PTI can be disabled safely.

Spectre variant 2, the system offers IBRS-based mitigation on Intel CPUs. The IBRS mitigation main disadvantage is the significant performance penalty. In OPNsense IBRS is enabled (for Intel) by default by disabling (0) `hw.ibrs_disable`, upstream

FreeBSD standard is disabled (1).

User/group separation (security.bsd)

Freebsd offers a couple of toggles to tighten security for ordinary users, these likely don't impact performance a lot, but these are the ones including descriptions that differ on our end (source `sysctl -d security.bsd`).

Setting	Description
security.bsd.hardlink_check_gid [0->1]	Unprivileged processes cannot create hard links t
security.bsd.hardlink_check_uid [0->1]	Unprivileged processes cannot create hard links t
security.bsd.unprivileged_proc_debug [1->0]	Unprivileged processes may use process debugg
security.bsd.see_other_gids [1->0]	Unprivileged processes may see subjects/objects
security.bsd.see_other_uids [1->0]	Unprivileged processes may see subjects/objects
security.bsd.unprivileged_read_msgbuf [1->0]	Unprivileged processes may read the kernel mes

Network

Netmap (IPS, Sensei, ...)

General

Netmap is a technology which enables fast packet processing while minimizing overhead, there are however some pitfalls which may turn your network interface unreachable.

Before using this technology, always make sure you have access via another interface (or console) to your firewall in case connectivity is dropped.

In order for netmap to function properly it is imperative that all sorts of driver / hardware acceleration is disabled (**Interfaces -> Settings**), this include `VLAN Hardware Filtering` as well (which wasn't disabled pre 20.7).

Some drivers have may have additional tunables, which enable hardware acceleration, make sure to disable them as well (.e.g intel ixl has `hw.ixl.enable_head_writeback`, which we disable by default)

Below you will find a list of tunables which are know to be (partial) incompatible with netmap.

Tunable	Description
hw.ixl.enable_head_writeback	Intel `ixl(4)` tunable for increased tx performance, OPN
dev.ax.<interface number>.sph_enable	AMD tunable to split header and payload into a separat

Decoupling Netmap from an application

It can be useful to split the functionality of Netmap and the application using it in order to determine whether it's Netmap or the application at fault for connectivity issues. To aid in this, Netmap's `bridge` utility has been added to our pkg repository for easy installation and use. To avoid ambiguity, it has been renamed to `netmap-bridge`.

You can install it by running `pkg install netmap-bridge` (`man netmap-bridge`).

`netmap-bridge` provides a L2 software bridge between two interfaces, but can also be used to bridge an interface and the host network stack like Suricata does. To replicate the behaviour of Suricata without actually running Suricata, run `netmap-bridge -i netmap:igb1`. Replace the interface as appropriate. While it is running, pass traffic as normal to determine if an original issue persists.

Common error codes

Any piece of software that uses system calls to communicate over sockets use the standard interface errno.h (`man errno`). If an error is logged, a return code is associated to a specific reason of failure. Some common ones are explained below:

XX	Name	Description
55	ENOBUFS	No buffer space available. An operation on a socket or pipe was not p `to`)
64	EHOSTDOWN	Host is down. A socket operation failed because the destination host
65	EHOSTUNREACH	No route to host. A socket operation was attempted to an unreachab

OpenVPN

Assigned Interfaces

While not strictly necessary, it is possible to assign individual interfaces for OpenVPN servers and clients alike. However doing so may yield unexpected behaviour of firewall rules. Most notably, rules created on an assigned interface of an OpenVPN Roadwarrior server are created with the `reply-to` directive by default, which breaks client connectivity.

Tip

In cases as described above, it can be observed that incoming traffic matches and passes the corresponding firewall rule, but reply traffic is never sent back to the connected client. This can be verified via the Web GUI by going to **Firewall -> Log Files -> Live View** and optionally by performing a packet capture on the affected interface.

There are multiple ways to fix this problem. For most setups, it will be sufficient to disable the automatically created IPv4 and IPv6 Gateways under **System -> Gateways -> Configuration**. Doing so will also disable the automatic addition of the `reply-to` directive to rules created on the interface, and client connectivity will be restored.

Another option is to manually select the option “Disable Reply-To” on each firewall rule you generate on the assigned interface. See Rules for further details.

The third option is to globally disable the generation of `reply-to` completely as described in (Advanced) Settings. However this method can break Multi-WAN setups.

Password reset

Sometimes people lose their passwords, in which case it can be practical to reset the root password without performing a reinstall.

This guide describes step by step how to reset the root password.

Note

You can also use the installer to reset the password, use the `Password Reset – Recover Installation` option in the installer after the keymap selection.

Step 1 : power

Since you need to influence the boot process to reset the password, you should start with powering off the (virtual) device.

Step 2 : console

Make sure you are able to access the (virtual) console, in case it's a physical machine you might want to connect a monitor and keyboard, when it's an appliance with serial access, make sure to connect to the serial/usb port using an application like putty.

Tip

When using devices from the OPNsense shop (https://shop.opnsense.com), usually there's a mini-usb to serial cable included in the box, the Serial Console connectivity guide will help you with the setup.

Step 3 : boot single

Switch the power and wait for the OPNsense splash screen to appear, the choose "Boot [S]single User" which should be option 2 in the list. (press 2)

Step 4: password reset

This stage depends on the type of installation, the root filesystem could either be UFS or ZFS, both requiring different commands to enable modifications in single user mode.

For UFS file systems, execute the following command in the shell to enable disk modifications:

```
/sbin/mount -o rw /
```

When using ZFS, the following commands are required

```
/sbin/mount -u /
/sbin/zfs mount -a
```

Now the disk can be written to, use the following commands to reset the password and reboot the machine:

```
opnsense-shell password
reboot
```

The *opnsense-shell* will ask for a confirmation, respond with *Y*

Performance

Receive-side scaling

Introduction

RSS is used to distribute packets over CPU cores using a hashing function – either with support in the hardware which offloads the hashing for you, or in software. The idea is to take as input the TCP 4-tuple (source address, source port, destination address, destination port) of a packet, hash this input using an in-kernel defined key, and selecting the resulting values' LSB as an index into a user-configurable indirection table. The indirection table is loaded into the hardware during boot and is used by the NIC to decide which CPU to interrupt with a given packet. All of this allows packets of the same origin/destination (a.k.a. flows) to be queued consistently on the same CPU.

Note

By default, RSS will be disabled since it's impact is quite far reaching.

Only enable this feature if you're interested in testing it and seeing if it will increase your throughput under high load – such as when using IDS/IPS.

Since we do not have every type of hardware available – nor the time to test all of them, no guarantee is given that a NIC driver will properly handle the kernel implementation or is even capable of using it.

The implementation of RSS is coupled with PCBGROUP – an implementation which introduces notions of CPU affinity for connections. While the latter will be of lesser importance for OPNsense, since it specifically applies to connections built up in userland using sockets (which is relevant to servers, not middleboxes), the idea of distributing work on a lower level with hardware support provides a myriad of benefits.

Network card / driver

Assuming you are using a modern NIC which supports multiple hardware queues and RSS, the configuration of a NIC will decide how and on which queue packets arrive on your system. This is also hardware dependent and will not be the same on every NIC. Should your driver support the option to enable/disable RSS, a sysctl tunable will be available.

It is possible for a NIC to perform RSS without being able to configure it. Should you wish to know if it can be enabled/disabled:

```
sysctl -a | grep rss
```

should show any drivers exposing the option via a tunable.

It is also possible that a driver does not expose this ability to the user, in which case you'd want to look up whether the NIC/driver supports RSS at all using online datasheets or a simple google search. For example, igb enables RSS by default, but does not reflect this in any configuration parameter. However, since it uses multiple queues:

```
# dmesg | grep vectors

igb0: Using MSI-X interrupts with 5 vectors
igb1: Using MSI-X interrupts with 5 vectors
igb2: Using MSI-X interrupts with 5 vectors
igb3: Using MSI-X interrupts with 5 vectors
```

It will most likely have some form of packet filtering to distribute packets over the hardware queues. In fact, igb does RSS by default.

For most NICs, RSS is the primary method of deciding which CPU to interrupt with a packet. NICs that do not implement any other type of filter and whose RSS feature is missing or turned off, will most likely interrupt only CPU 0 at all times – which will reduce potential throughput due to cache line migrations and lock contention. Please keep system-wide RSS disabled if this is the case.

The last but not least thing to consider is the fact that driver support with the in-kernel implementation of RSS is a must. Proper driver support will ensure the correct key and indirection table being set in hardware. Drivers which support RSS according to the source code (but mostly untested):

- em
- igb -> tested & working
- axgbe -> tested & working
- netvsc
- ixgbe
- ixl
- cxgbe
- lio
- mlx5
- sfxge

Kernel support

Internally, FreeBSD uses netisr as an abstraction layer for dispatching packets to the upper protocols. Within the implementation, the default setting is to restrict packet processing to one thread only. Since RSS now provides a way to keep flows local to a CPU, the following sysctls should be set in `System->Settings->Tunables`:

- net.isr.bindthreads = 1
- net.isr.maxthreads = -1

The first option causes threads to be bound to a CPU, the next one assigns a workstream to each core available.

Furthermore, the RSS implementation also provides a few necessary sysctls:

- net.inet.rss.enabled = 1

Makes sure RSS is enabled. This is disabled by default to prevent regressions on NICs that do not properly implement the RSS interface.

- net.inet.rss.bits = X

This one is dependent on the amount of cores you have. By default the amount of bits here represent the amount of cores x 2 in binary. This is done on purpose to provide load-balancing, though there is no current implementation for this so we recommend setting this value to the amount of bits representing the number of CPU cores. This means we use the following values:

- for 4-core systems, use '2'
- for 8-core systems, use '3'
- for 16-core systems, use '4'
- Etc.

Note

Remember to reboot the machine after changing `net.inet.rss.enabled` as this option can only be set during boot.

Validate settings

If RSS is enabled with the 'enabled' sysctl, the packet dispatching policy will move from 'direct' to 'hybrid'. This will directly dispatch a packet on the current context when allowed, otherwise it will queue the packet on the bound CPU on which it came in on. Please note that this will increase the interrupt load as seen in 'top -P'. This simply means that packets are being processed with the highest priority in the CPU scheduler - it does not mean the CPU is under more load than normal.

The correct working of netisr can be verified by running:

```
netstat -Q
```

Note regarding IPS

When Suricata is running in IPS mode, Netmap is utilized to fetch packets off the line for inspection. By default, OPNsense has configured Suricata in such a way that the packet which has passed inspection will be re-injected into the host networking stack for routing/firewalling purposes. The current Suricata/Netmap implementation limits this re-injection to one thread only. Work is underway to address this issue since the new Netmap API (V14+) is now capable of increasing this thread count. Until then, no benefit is gained from RSS when using IPS.

Reset firmware configuration

In cases where the firmware configuration is corrupt in the `config.xml` file, we can reset that section of our configuration using the following command (executed on a console or via SSH):

```
pluginctl -f system.firmware
```

Which shows the current data stored and asks for removal, choose `Y` here to drop that part and circle back to the **System ▸ Firmware** section to store the configuration again.

Note

If our factory left a cache file which keeps a “factory-” plugin visible at all times, you can safely remove the reference using the following command:

`rm /usr/local/opnsense/version/factory-*`

WebGui access reset

If for some reason the webgui certificate is broken, you can reconfigure access using the console menu. Select `Set interface IP address` (option 2) from the menu, reconfigure an interface, after providing the address configuration you can either (temporary) switch back to `HTTP` or in the next step generate a new self-signed certificate.

It is also possible to reset the defaults in the final step ("**Restore web GUI access defaults?**"), in case something went wrong while setting up anti lockout policies or after changing interfaces.

Tip

When logged in directly via a console or shell, you can also use the following command to generate a new self-signed certificate and restart the web ui:

```
configctl webgui restart renew
```

Initial Installation & Configuration

Note

Just looking for how to invoke the installer? When the live environment has been started, log in with user **installer** and password **opnsense**.

Index

- Initial Installation & Configuration
  - Architecture
  - Embedded vs Full
  - Installation Images
  - Image Filename Composition
  - Download and Verification
  - Installation Media
  - System Boot Preparation
  - Installation Instructions
    - OPNsense Importer
    - Live Environment
    - OPNsense Installer
    - Nano Image
  - Initial Configuration

Architecture

The **software setup** and installation of OPNsense® is available for the x86-64 microprocessor architecture only.

Embedded vs Full

OPNsense offers two Image types with all major releases: embedded and full images. The Embedded Image is intended for environments where preinstalling the storage media is required due to a lack of local resources on the firewall like storage, and/or console access (VGA/Serial). The image is tailored to reduce write cycles as well, but

the image can be used anywhere. Another reason for the Embedded Image is to eliminate the need for local console access for installing OPNsense. Installation is managed by prewriting the image to a storage device, installing the storage device, and booting the system.

Full Images provide installation tools like OPNsense Importer, Live Environment, and Installer. Full Images are released to support different console/hardware installation requirements.

Both image types can be installed and run from virtual disks (VM), SD memory cards, USB disks, solid-state disks (SSD), or hard disk drives (HDD).

The main differences between embedded and full images are:

Embedded	Full
Writes to RAM disk	Writes to local disk
No log data retention after reboot	Log data retention after reboot
Not intended for local disk writes	Suitable for disk writes.
Embedded only use, SWAP file is optional	Can enable RAM disk for embedded mode.

The embedded image stores logging and cache data in memory only, while full versions will keep the data stored on the local drive. A full version can mimic the behavior of an embedded version by enabling RAM disks, which is especially useful for SD memory card installations.

Warning

See the chapter Hardware Sizing & Setup for further information on hardware requirements prior to an install.

Installation Images

Depending on your hardware and use case, different installation options are available:

Type	Description
dvd	ISO image boots into a live environment in VGA-only mode with UEFI support
vga	USB image boots into a live environment in VGA-only mode with UEFI support

Type	Description
serial	USB image boots into live environment running in serial console (115200) mode only with U
nano	Image for preinstalling onto >=4 GB USB drives, SD, or CF cards for use with embedded de

Note

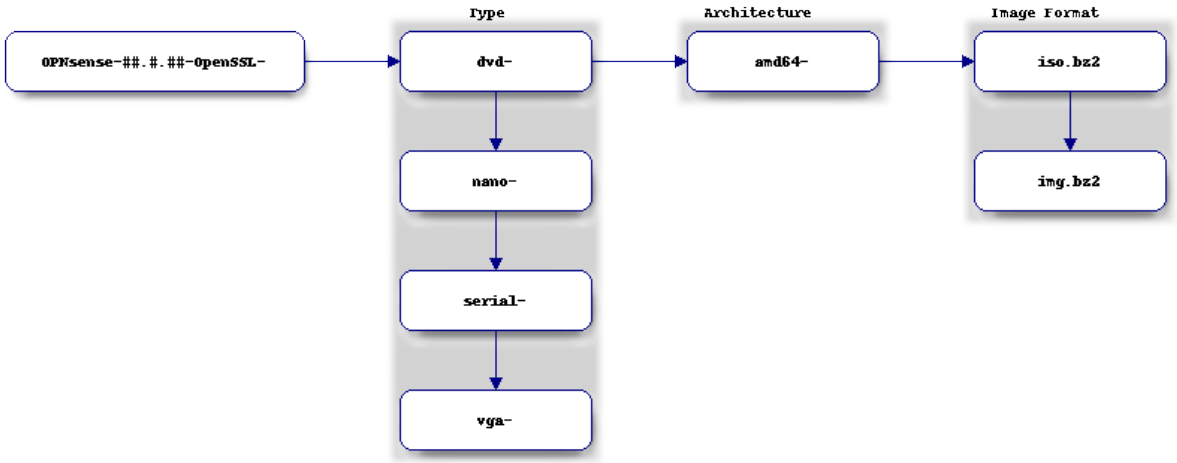
All Full Image types can run both OPNsense Importer before booting into the Live environment and also run Installer once booted into the Live environment.

Warning

Flash memory cards will only tolerate a limited number of writes and re-writes. For the Nano image, **/var/log** and **/tmp** are memory disks by default to prolong the lifetimes of CF/SD cards and SSDs in special cases.

To enable non-embedded versions: Go to **System ▸ Settings ▸ Miscellaneous ▸ Disk / Memory Settings**, change the setting, then reboot. Consider enabling an external syslog server as well.

Image Filename Composition



Note

Please be aware that the latest installation media does not always correspond with the latest released version available. OPNsense installation images are provided on a scheduled basis with major release versions in January and July. More

information on our release schedule is available from our package repository, see README. We are encouraged to update OPNsense after installation to be on the latest release available, see Update Page.

Download and Verification

The OPNsense distribution can be downloaded from one of our mirrors.

OpenSSL is used for image file verification. 4 files are needed for verification process:

- The SHA-256 checksum file (<filename>.sha256)
- The bzip-compressed image file (<filename>.<image>.bz2)
- The signature file for the uncompressed image file (<filename>.<image>.sig)
- The OpenSSL public key (<filename>.pub)

Use one of the OPNsense mirrors to download these files:

1. Go to the bottom of OPNsense download page.
2. Click one of the available mirrors closest to your location.
3. Download one of each file mentioned above for your Image type.

The OpenSSL public key (.pub) is required to verify against. Although the file is available on the mirror's repository, you should not trust the copy there. Download it, open it up, and verify the public key matches the one from other sources. If it does not, the mirror may have been hacked, or you may be the victim of a man-in-the-middle attack. Some other sources to get the public key from include:

- https://pkg.opnsense.org/releases/mirror/README
- https://forum.opnsense.org/index.php?board=11.0
- https://opnsense.org/blog/
- https://github.com/opnsense/changelog/tree/master/community
- https://pkg.opnsense.org (/<FreeBSD:<version>:<architecture>/<release version>/sets/changelog.txz)

Note

Only major release announcements for images contain the public key, and update release announcements will not. i.e. 22.1 will have a copy of the public key in the release announcement, but 22.1.9 will not.

Once you download all the required files and verify that the public key matches the public key found in one of the alternate sources listed above, you can be relatively confident that the key has not been tampered with. To verify the downloaded image, run the following commands (substituting the filenames in brackets for the files you downloaded):

```
openssl sha256 OPNsense-<filename>.bz2
```

Match the checksum command output with the checksum values in the file `OPNsense-<version>-OpenSSL-checksums-amd64.sha256`. If the checksums don't match, redownload your image file.

If the checksums match, continue with the verification commands.

```
openssl base64 -d -in OPNsense-<filename>.<image>.sig -out /tmp/image.sig
openssl dgst -sha256 -verify OPNsense-<filename>.pub -signature /tmp/image.sig OPNsense-<filename>.<image>
```

Warning

Make sure to unpack the image using `bunzip2` before verifying. Our signatures are generated before compressing them (as of OPNsense version 24.1)

If the output of the second command is "**Verified OK**", your image file was verified successfully, and it is safe to install from it. Any other outputs, and you may need to check your commands for errors, or the image file may have been compromised.

Installation Media

Now that you have downloaded and verified the installation image from above, you must unpack the image file before you can write the image to disk. For Unix-like OSes use the following command:

```
bzip2 -d OPNsense-<filename>.bz2
```

For Windows use an application like 7zip. The `.bz2` will be removed from the end of the filename after command/application completes.

After unpacking the image you can create the installation media. The easiest method to install OPNsense is to use the USB "vga" Image. If your target platform has a serial console interface choose the "serial" image. If you need to know more about using the serial console interface, consult the serial access how-to.

Write the image to a USB flash drive (>=1 GB) or hard disk, using either dd for Unix-like OSes or, for Windows, physdiskwrite, Etcher, or Rufus.

FreeBSD

```
dd if=OPNsense-##.#.##-[Type]-[Architecture].img of=/dev/daX bs=16k
```

Where X = the device number of your USB flash drive (check `dmesg`)

OpenBSD

```
dd if=OPNsense-##.#.##-[Type]-[Architecture].img of=/dev/rsd6c bs=16k
```

The device must be the ENTIRE device (in Windows/DOS language: the 'C' partition), and a raw I/O device (the 'r' in front of the device "sd6"), not a block mode device.

Linux

```
sudo dd if=OPNsense-##.#.##-[Type]-[Architecture].img of=/dev/sdX bs=16k
```

where X = the IDE device name of your USB flash drive (check with hdparm -i /dev/sdX) (ignore the warning about trailing garbage - it's because of the digital signature)

macOS

```
sudo dd if=OPNsense-##.#.##-[Type]-[Architecture].img of=/dev/rdiskX bs=64k
```

where r = raw device, and where X = the disk device number of your CF card (check Disk Utility) (ignore the warning about trailing garbage - it's because of the digital signature)

Windows

```
physdiskwrite -u OPNsense-##.#.##-[Type]-[Architecture].img
```

(use v0.3 or later!)

System Boot Preparation

After preparing the installation media, we need to make sure we can access the console (either via keyboard and [virtual]monitor or serial connectivity). Next we need to know how to access the boot menu or the system BIOS (UEFI) to boot from the installation media. In most cases, it will be a function (F#), Del, or ESC key that needs to pressed immediately after powering on (or rebooting) the system. Usually within the first 2 to 3 seconds from powering up.

Tip

OPNsense devices from the OPNsense shop use `<ESC>` to enter the BIOS and boot selection options.

Note

Serial connectivity settings for DECXXXX devices can be found here

Installation Instructions

Install Instructions

OPNsense installation boot process allows us to run several optional configuration steps. The boot process was designed to always boot into the live environment, allowing us to access the GUI or even SSH directly. If a timeout was missed, restart the boot procedure.

OPNsense Importer

All Full Images have the OPNsense Importer feature that offers flexibility in recovering failed firewalls, testing new releases without overwriting the current installation by running the new version in memory with the existing configuration or migrating configurations to new hardware installations. Using Importer is slightly different between previous installs with existing configurations on disk vs new installations/migrations.

For systems that have OPNsense installed and the configuration intact, here is the process:

1. Boot the system with installation media
2. Press any key when you see **"Press any key to start the configuration importer"**.
    1. If you see the login prompt, you have passed the importer and will need to reboot.
3. Type the device name of the existing drive that contains the configuration and press enter.
4. If Importer is successful, the boot process will continue into the live environment using the stored configuration on disk.
5. If Importer was unsuccessful, we will be returned to the device selection prompt. Confirm the device name is correct and try again. Otherwise, there may be possible disk corruption and restoring from backup.

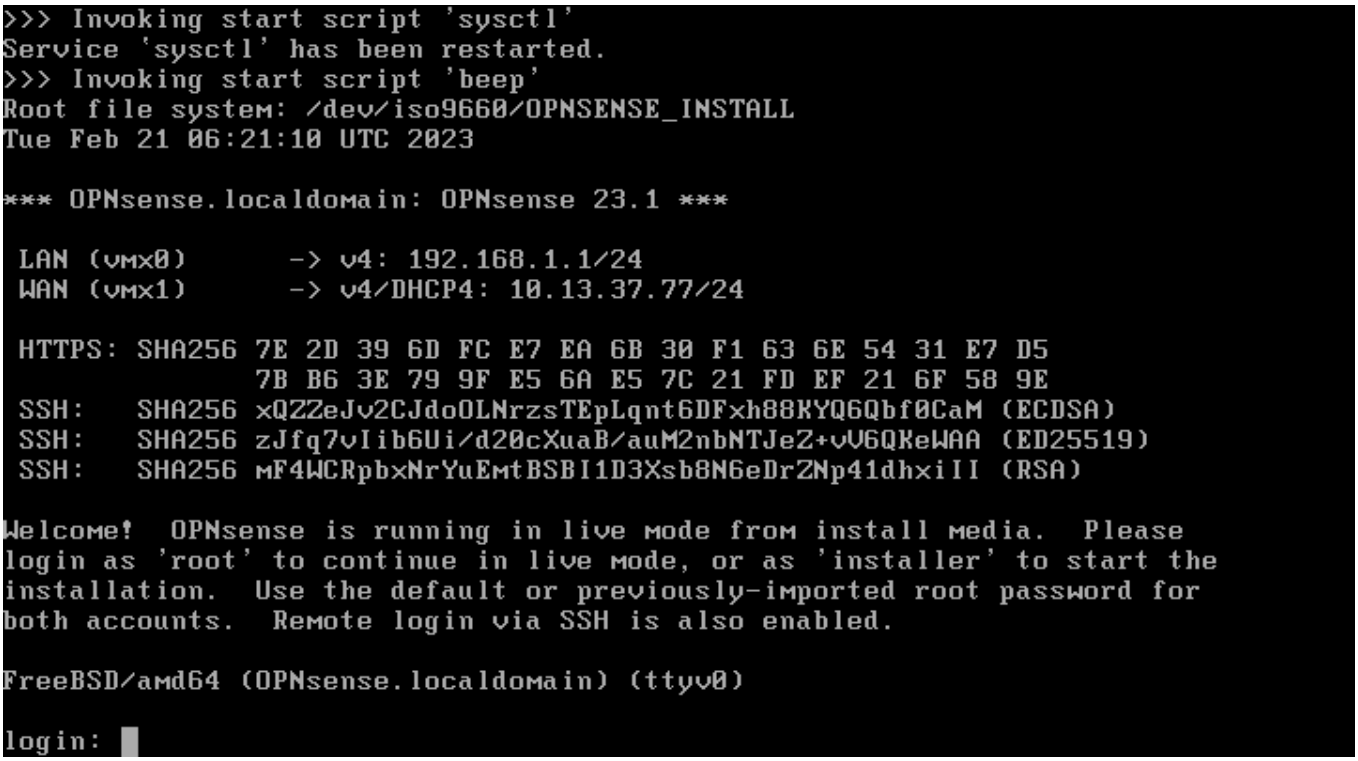
At this point, the system will boot up with a fully functional firewall in live environment using existing configuration but will not overwrite the previous installation. Use this feature for safely previewing or testing upgrades.

For new installations or migrations, follow this process:

1. We must have a 2nd USB drive formatted with FAT or FAT32 File system.
    1. Preferable non-bootable USB drive.
2. Create a **conf** directory on the root of the USB drive
3. Place an *unencrypted* <downloaded backup>.xml into /conf and rename the file to **config.xml** (`/conf/config.xml`)
4. Put both the Installation media and the 2nd USB drive into the system and power up / reboot.
5. Boot the system from the OPNsense Installation media via Boot Menu or BIOS (UEFI).
6. Press any key when you see: **"Press any key to start the configuration importer"**

7. Type the device name of the 2nd USB Drive, e.g. *da0* or *nvd0* , and press Enter.
    1. If Importer is successful, the boot process will continue into the Live environment using the configuration stored on the USB drive.
    2. If unsuccessful, the importer will error and return to the device selection prompt. In this case, repeat steps 1-3 again.

Live Environment



After booting with an OPNsense Full Image (DVD, VGA, Serial), the firewall will be in the Live environment with and without the use of OPNsense Importer. We can interact with the Live environment via Local Console, GUI (HTTPS), or SSH.

By default, we can log into the shell using the user `root` with the password `opnsense` to operate the live environment via the local console.

The GUI is accessible at https://192.168.1.1/ using Username: `root` Password: `opnsense` by default (unless a previous configuration was imported).

Using SSH we can access the firewall at IP **192.168.1.1** . Both the **root** and **installer** users are available with the password specified above.

Note

The installation media is read-only, which means your current live configuration will be lost after reboot.

Continue to OPNsense Installer to install OPNsense to the local storage device.

OPNsense Installer

Note

To invoke the installer, log in with user **installer** and password **opnsense**

After successfully booting up with the OPNsense Full Image (DVD, VGA, Serial), the firewall will be at the Live Environment's login: prompt. To start the installation process, login with the user `installer` and password `opnsense`. If Importer was used to import an existing configuration, the installer and root user password will be the root password from the imported configuration.

If the installer user does not work, log in as user root and select: `8) Shell` from the menu and type `opnsense-installer`. The `opnsense-importer` can also be run this way should you need to rerun the import.

The installer can always be run to clone an existing system, even for Nano images. This can be useful for creating live backups for later recovery.

Tip

The installer can also be started from an internal host using SSH. The default IP address is `192.168.1.1`

Attention

When installing an appliance with SD or MMC flash card storage, ensure you select the UFS filesystem and choose `mmcsd0` in the Disk Selection. Do not choose `da0` as that is the USB drive. As an example, this is applicable to the DEC677 Desktop Security Appliance.

The installation process involves the following steps:

1. Keymap selection - The default configuration should be fine for most occasions.

2. Install (UFS|ZFS) - Choose UFS or ZFS filesystem. ZFS is in most cases the best option as it is the most reliable option, but it does require enough capacity (a couple of gigabytes at least).
3. Partitioning (ZFS) - Choose a device type. The default option (stripe) is usually acceptable when using a single disk.
4. Disk Selection (ZFS) - Select the Storage device e.g. `da0` or `nvd0`
5. Last Chance! - Select Yes to continue with partitioning and to format the disk. However, doing so will **destroy** the contents of the disk.
6. Continue with recommended swap (UFS) - Yes is usually fine here unless the install target is very small (< 16GB)
7. Select Root Password - Change and confirm the new root password
8. Select Complete Install - Exits the installer and reboots the machine. The system is now installed and ready for initial configuration.

Warning

You will lose all files on the installation disk. If another disk is to be used then choose a Custom installation instead of the Quick/Easy Install.

Nano Image

To use the nano image follow this process:

1. Create the system disk by using the Nano image. See Installation Media how to write the nano image to disk.
2. Install the system disk drive into the system.
3. Configure the system (BIOS) to boot from this disk.
4. After the system boots, the firewall is ready to be configured.

Using the nano image for embedded systems, your firewall is already up and running. The configuration settings to enable Memory Disks (RAM disks) that minimize write cycles to relevant partitions by mounting these partitions in system memory and reporting features are disabled by default.

Initial Configuration

After installation, the system will prompt you for the interface assignment. If you ignore this, then the default settings will be applied. Installation ends with the login prompt.

By default you have to log in to enter the console.

Welcome message

```
* * * Welcome to OPNsense [OPNsense 15.7.25 (amd64/OpenSSL) on OPNsense * * *

WAN (em1)     -> v4/DHCP4: 192.168.2.100/24
LAN (em0)     -> v4: 192.168.1.1/24

FreeBSD/10.1 (OPNsense.localdomain) (ttyv0)

login:
```

Tip

A user can login to the console menu with his credentials. The default credentials after a fresh install are username “root” and password “opnsense”.

VLANs and assigning interfaces

If you choose to do manual interface assignment or when no config file can be found then you are asked to assign Interfaces and VLANs. VLANs are optional. If you do not need VLANs then choose **no**. You can always configure VLANs at a later time.

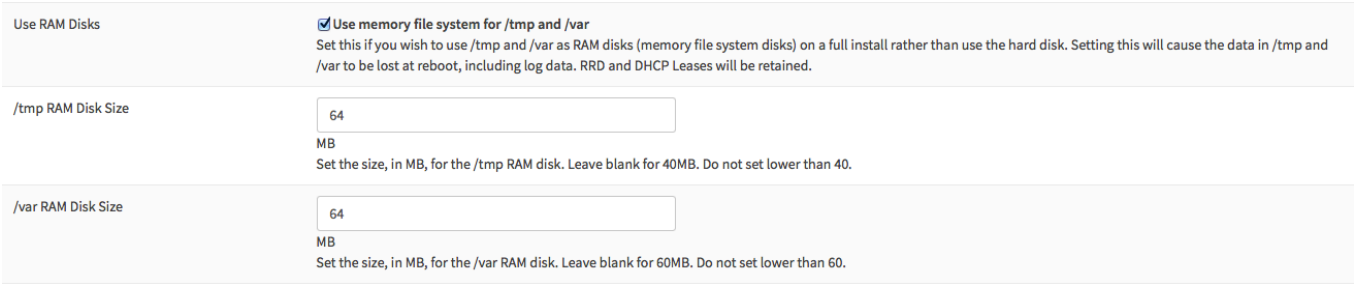
LAN, WAN and optional interfaces

The first interface is the LAN interface. Type the appropriate interface name, for example “em0”. The second interface is the WAN interface. Type the appropriate interface name, e.g. “em1” . Possible additional interfaces can be assigned as OPT interfaces. If you assigned all your interfaces you can press [ENTER] and confirm the settings. OPNsense will configure your system and present the login prompt when finished.

Minimum installation actions

In case of a minimum install setup (i.e. on CF cards), OPNsense can be run with all standard features, except for the ones that require disk writes, e.g. a caching proxy like Squid. Do not create a swap slice, but a RAM Disk instead. In the GUI enable **System ▸ Settings ▸ Miscellaneous ▸ RAM Disk Settings** and set the size to 100-128 MB or more, depending on your available RAM. Afterwards reboot.

Enable RAM disk manually



Then via console, check your /etc/fstab and make sure your primary partition has **rw,noatime** instead of just **rw**.

Console

The console menu shows 13 options.

```
0)     Logout                              7)     Ping host
1)     Assign interfaces                   8)     Shell
2)     Set interface(s) IP address         9)     pfTop
3)     Reset the root password             10)    Filter logs
4)     Reset to factory defaults           11)    Restart web interface
5)     Reboot system                       12)    Upgrade from console
6)     Halt system                         13)    Restore a configuration
```

Table: *The console menu*

opnsense-update

OPNsense features a command line interface (CLI) tool "opnsense-update". Via menu option **8) Shell**, the user can get to the shell and use opnsense-update.

For help, type *man opnsense-update* and press [Enter].

Upgrade from console

The other method to upgrade the system is via console option **12) Upgrade from console**

GUI

An update can be done through the GUI via **System ‣ Firmware ‣ Updates** .

Status | Settings | Changelog | Updates | Plugins | Packages

Type	opnsense
Version	25.1
Architecture	amd64
Commit	da994c043
Mirror	https://pkg.opnsense.org/FreeBSD:14:amd64/25.1
Repositories	OPNsense (Priority: 11)
Updated on	Thu Jun 5 16:55:24 CEST 2025
Checked on	N/A

Check for updates | Run an audit

Updates

The OPNsense update schedule consists of two major releases each year, which receive minor updates about every two weeks. The major release version number consists of the year and month of release (e.g. 25.1 for the January 2025 release), with the fortnightly minor updates adding a third number (e.g. 25.1.3 for the third update to 25.1).

Update settings

By navigating to **System ▸ Firmware ▸ Settings**, you can influence the firmware update settings:

- **Firmware Mirror:** this influences where OPNsense tries to get its updates from. If you have trouble updating or searching for updates, or if your current mirror is running slowly, you can change it here.
- **Release Type:** this switches between the regular fortnightly schedule of tested releases (Production) or the newest, not fully tested code (Development). **Please leave this setting on "Production", unless you fully understand the implications of switching.**

Tip

The settings page is also the place where you can run audits which help in debugging common connectivity issues. Just press "Run an audit" and choose "Connectivity" from the list if you encounter issues with the remote server.

Minor updates

Updates can be installed from the web interface by going to **System ▸ Firmware ▸ Status**. On this page, you can click **Check for updates** to search for updates. If they are available, a button will appear asking to install them.

Major Upgrades

Major upgrades are recommended to be done via a VGA display or serial console so you can see what is going on.

Note

You can find documentation about serial access under Serial Access

Warning

Major updates are installed offline. This means no web or SSH server is running for you to monitor the upgrade. If something fails, you will need a secondary connection or direct access to revert to a snapshot or repair the installation.

If you choose option 12 on the console menu on latest release, you are asked if you want to upgrade to the newest version or to the next major release. Type in the major release number (for example “19.1”) and press enter. OPNsense will download all release files for an offline upgrade (kernel, packages etc.) and reboot afterwards.

After the reboot, it will install all updates and when it is done, it will reboot again. You should then be on the desired release.

Firmware

In the firmware section you can keep your OPNsense up to date and have the ability to install additional software.

Updates

How to install updates and information about our versioning can be found in the Installation and setup section, which can be found in the Updates document.

Plugins

Plugins are additional software packages that are available for OPNsense, usually they come with their own frontend components to setup the software underneath. Shown by default are support tier 1 and 2 plugins.

Here you can also find community plugins from lower tiers, such as **bind**, **c-icap**, **freeradius** and others. These are not listed by default. To view them use the **Show community plugins** checkbox.

When on the development version, a **-devel** version is also available available, which may contain features still under development or ready for wider testing before their eventual release (master branch on GitHub versus release).

Our plugin support tiers are explained in detail in the Support Options document.

Note

You can use the + symbol to install the package.

Tip

Use the info button to display information about the package and to find the active maintainer of this piece of software.

Packages

The packages tab contains the installed ports packages, here you can check licenses, force reinstalls or lock versions.

Changelog

If you would like to read about the changes in (past) releases, you can do so in the **Changelog** tab.

Status (firmware audits)

Audits provided are troubleshooting tools for a variety of purposes explained below. The forum is a good place to post your update and upgrade issues if you get stuck despite the best effort.

The cleanup audit triggers an immediate removal of all tempporary update files. Sometimes these files are not cleaned up correctly and can cause issues with the next round of package updates.

The connectivity audit offers a direction where to look for issues during updates and the following causes are in our experience most common:

- Misconfigured DNS settings, check **System ‣ Settings ‣ General** for configured servers the firewall is allowed to use
- Misconfigured IPv6, in which case "Prefer IPv4 over IPv6" in **System ‣ Settings ‣ General** might help to prevent the system from using IPv6 in these cases
- In HA (CARP) setups, using the wrong external IP address, usually caused by a misconfigured Source NAT rule, is easy to check by disabling manual Source NAT rules in **Firewall ‣ NAT ‣ Source NAT (Outbound)**.

The heath audit can also help with uncovering installation and disk / file system issues. Additionally, major ugpgrades may not pass certain sanity checks that need to be corrected first which may include the command line:

- "Could not determine core package name." can indicate that the local package manager database was lost. See *opnsense-bootstrap* command to fix.
- "No package manager is installed to perform upgrades." can indicate a broken installation. Try to reinstall the "pkg" package via *System –> Firmware –> Packages*.

- "The Package manager is incompatible and needs a reinstall." can indicate misuse of the FreeBSD repository. Try to reinstall the "pkg" package via *System –> Firmware –> Packages*.
- "Core package not known to package database." can mean that the mirror settings are wrong, the main mirror no longer holds any packages or that the mirror is unreachable for other reasons.

The security audit checks each installed package's version against a database of known vulnerabilities. Vulnerabilities are reported for triage and inspection, not for reporting them to OPNsense. The appearance of vulnerabilities in an install indicate that the next update will likely address them.

After having performed a major upgrade, the upgrade audit shows the package upgrade log for further inspection. This can be helpful to identify package conflicts that led to partial or full upgrade failures.

Settings

The settings menu contains all available mirrors and options which you can choose for your installation. Usually the default options are good enough here, but if you want to choose a mirror more close to home you can do so here.

Tip

The settings menu also provides the option to test development versions, which can be practical when testing features that are planned for release. Just change the release type to **Development**.

Note

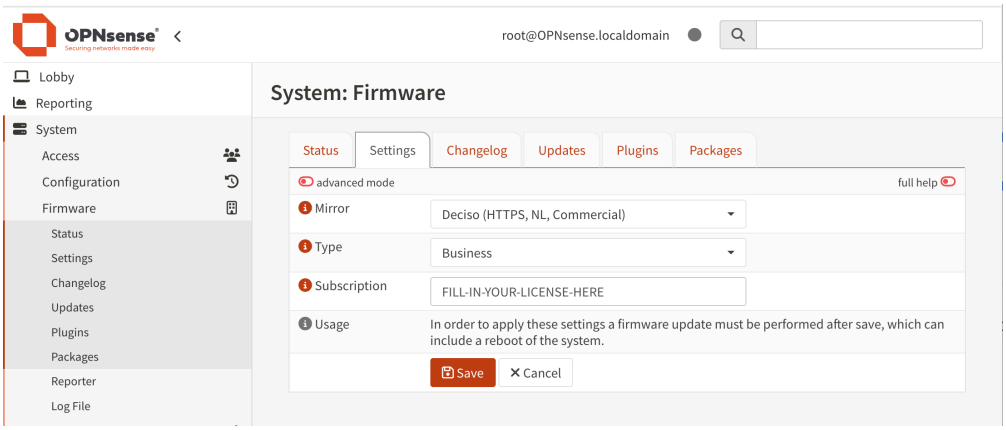
Although OPNsense does not support the configuration of a proxy server, for services like the firmware updater it is possible to add these settings manually in our `configd` service. For more information we refer to the envronment section in our development docs.

Apply the Business Edition

When you have purchased a license for the Business Edition or received it pre-installed on an appliance, you will have to enable the license first.

In order to do so, please choose the following settings:

Mirror:	Deciso (HTTPS,NL,Commercial)
Flavour:	OpenSSL
Type:	Business
Subscription:	XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX (the activation key for the product)



After save, go back to the status tab and click **Check for updates**

Note

Upgrading to OPNsense BE is only possible when the installed community version number is lower than the last available business edition. E.g. you can upgrade **22.7.x** to **22.10.x**, but you can not upgrade **23.1** to **22.10**. You can always re-install using the installer found on the business mirror

Configuration

The configuration section contains some tools to keep track of your setup.

Backup

You can backup and restore your machines configuration using the **System ▸ Configuration ▸ Backups** section. When performing a local backup, you can choose to protect it with a password and decide whether or not you would like to store rrd statistics as well (in which case they will be stored in the same export xml file).

Warning

The configured console settings of a machine may not be applicable to the situation of other (virtual) hardware platforms. To avoid losing console access to the firewall in these cases, the configuration restore will not apply the backups' console settings by default. To disable this behaviour, uncheck **Exclude console settings from import**.

Note

Due to the sensitive nature of your configuration data, its advisable to protect backups with a strong password when storing them or distributing them to others.

Note

RRD statistics are used in System Health

Restoring backups can either be performed partially or for the complete configuration. Since configurations usually have various components that depend on each other, it's most safe to restore a complete configuration.

Warning

Partial restores can lead to unexpected behavior, use with care. Future versions might not support this feature due to consistency reasons. (not all components can be partially exported)

Note

If restoring a backup on a different (virtual) machine and the hardware interface names do not match, a reboot will not automatically take place, instead you will be prompted to fix the interface assignments.

- Backups via secure copy (sftp)
- Traceability of configuration changes using Git
- Cloud Backup / Nextcloud

Defaults

With the defaults tool you can reset your firewall back to firmware defaults and shutdown when done.

History

The history menu helps you track changes between modifications and offer the opportunity to download older versions of your settings.

When OPNcentral is installed on the firewall and backups are being performed, there will be a host selector at the top of the page to select which host should be inspected.

Each backup preserved on this machine will be presented as an item in the dropdown, when selecting an item, the previous backup (when available) will be selected automatically as comparison target. The final pane on the screen contains the changes between both selected versions in unified diff format.

The following buttons are available in the “backups (compare)” pane:

- Revert to this backup (this firewall only), keep in mind no services will be restarted after this action, for a clean state a reboot might be required
- Remove this backup (this firewall only)
- Download the selected backup (in left dropdown)

Tip

You can specify the number of backups to keep in the backups menu (**System** ▸ **Configuration** ▸ **Backups**), which can be quite practical when a higher level of auditability is required.

Interface configuration

All traffic in OPNsense travels via interfaces. By default, WAN and LAN are assigned, but many more are possible, like GUESTNET (captive portal) and PFSYNC (high availability).

Interface settings are now managed through an MVC/API implementation integrated into the grid under **Interfaces ▸ Assignments**. Commonly required advanced DHCP options that were not part of the legacy basic mode are available when advanced mode is enabled in the assignment dialog.

The legacy advanced and file-based DHCP modes are not supported by the MVC/API implementation. Saving an interface in the Assignments dialog removes those settings. The legacy `/interfaces.php` page remains available by entering its URL directly and retains the old-style settings. It is expected to move to a plugin for OPNsense 27.1.

Saving operational settings queues them for reconfiguration and permanent storage. Select **Apply** to process the queue using the rewritten backend sequence, which otherwise behaves similarly to applying changes on the legacy page. Pending changes are held in temporary storage; rebooting without applying them discards them.

Note

For legacy compatibility WAN interfaces set to type DHCP or interfaces with a *Gateway Rules* selection send reply packets to the corresponding gateway directly, also when the sender is on the same interface. This will break connectivity in some rare scenarios and can be disabled via **Firewall->Settings->Advanced->Disable reply-to**.

Assignments

Most interfaces have to be assigned to a physical port. By default, LAN is assigned to port 0 and WAN is assigned to port 1. Go to **Interfaces ▸ Assignments** to manage both assignments and their configuration. The grid lists every assigned interface and its device, address modes and status. Use the edit button on a row to change an existing interface. Use the add button to assign and configure an available device. Save changes in the dialog, then select **Apply** on the Assignments page to activate all pending interface changes.

Note

Changing an interface's device or address configuration can interrupt connectivity. The description and lock fields are saved immediately; operational changes remain pending until they are applied.

The edit dialog groups the common options as follows. Advanced fields are shown when advanced mode is enabled.

Option	Explanation
Basic configuration	
Identifier	The internal interface name, such as `wan`, `lan` or `opt1`. This value is
Enabled	Enable or disable the interface without removing its assignment.
Lock	Prevent accidental removal. Clear this option and save before deletin
Description	A short name used to identify the interface throughout the user interf
Device configuration	
Device	The physical or virtual device assigned to this interface. A device can
Promiscuous mode	Permanently receive all packets visible to the device. Leave disabled
MAC address (spoof)	Override the device MAC address. This advanced option normally ren
MTU	Maximum Transmission Unit. Leave empty to use the device default.
MSS	Enable TCP MSS clamping to the entered value minus the IPv4 or IPv
Hardware settings	
Media (speed and duplex)	Leave at the default unless the selected value is known to match the
Overwrite global settings	Use the hardware CRC, TSO, LRO and VLAN filtering values below ins

Option	Explanation
Hardware CRC	Disable checksum offloading for this interface.
Hardware TSO	Disable TCP segmentation offloading for this interface.
Hardware LRO	Disable large receive offloading for this interface.
VLAN Hardware Filtering	Enable, disable or retain the device default for VLAN hardware filteri
Generic configuration	
Block private networks	Block traffic from RFC 1918, loopback, link-local and carrier-grade NA
Block bogon networks	Block traffic from reserved or unallocated source prefixes that should

Note

When configuring VPN clients without static tunnel addresses, you can use the "Dynamic gateway policy" option to automatically generate gateways to the device (without address).

The address fields shown in the dialog depend on the selected **IPv4 Configuration Type** and **IPv6 Configuration Type**.

For IPv4:

Option	Explanation
IPv4 Configuration Type	Select `None`, `Static IPv4`, `DHCP` or the mode matching an assigned PI
Static IPv4	
IPv4 address	Enter the address and prefix length in CIDR notation, for example `192.`
IPv4 gateway rules	Select a gateway belonging to this interface. Reply-to policy and, in Au
DHCP	
Alias IPv4 address	Fixed alias address used by the DHCP client.
Alias IPv4 subnet mask	Prefix length for the alias address.
Reject Leases From	Ignore offers from the listed DHCP servers, for example an ISP modem
Hostname	Client identifier and hostname sent with the DHCP request. Some ISPs
Honour MTU	Allow the DHCP-provided MTU to be applied. This advanced option is c
Use VLAN priority	Apply the selected 802.1p priority to DHCPv4 requests when required b
Send Options	Advanced DHCPv4 options sent to the server, one per line.

Option	Explanation
Request Options	Advanced DHCPv4 options requested from the server, one per line.

For IPv6:

Option	Explanation
IPv6 Configuration Type	Select `None`, `Static IPv6`, `DHCPv6`, `Link-local`, `SLAAC`, a tunnel or
Static IPv6	
IPv6 address	Enter the address and prefix length in CIDR notation.
IPv6 gateway rules	Select an IPv6 gateway belonging to this interface to generate the
DHCPv6	
Use VLAN priority	Apply the selected 802.1p priority to DHCPv6 requests when require
Prefix delegation size	Delegated prefix length supplied by the ISP, or `None` when no prefix
Request prefix only	Request a delegated prefix without requesting an address.
Information Only	Advanced stateless mode that requests configuration parameters b
Request DNS configuration	Request DNS server information from the DHCPv6 server.
Send prefix hint	Indicate the desired delegation size to the DHCPv6 server.
Send rapid commit	Request a two-message rapid-commit exchange instead of waiting
Optional prefix ID	Hexadecimal subnet ID to select from the delegated prefix.
Optional interface ID	Hexadecimal value used for the lower interface-identifier portion of
Optional prefix IAID	Non-standard identity-association ID for the prefix request, when r
Send Options	Advanced DHCPv6 options sent to the server, one per line.
Request Options	Advanced DHCPv6 options requested from the server, one per line.
Link-local	Generate an automatic link-local address (LLA) based on the MAC
SLAAC	Same as link-local mode, but received router advertisements (RA) w
6RD Rapid Deployment	
6RD prefix	The 6RD IPv6 prefix assigned by your ISP. e.g. '2001:db8::/32'
6RD Border Relay	The 6RD IPv4 gateway address assigned by your ISP
6RD IPv4 Prefix length	The 6RD IPv4 prefix length. Normally specified by the ISP. A value of
6RD IPv4 Prefix address	The 6RD IPv4 prefix address. Optionally overrides the automatic de
Identity association	*Track Interface (legacy)*

Option	Explanation
Parent interface	Select the dynamic IPv6 interface whose delegated prefix should be
Assign prefix ID	The hexadecimal /64 network ID selected from the delegated prefix
Reserved prefix range	Length of the range reserved for downstream delegation. The rang
Optional interface ID	Hexadecimal value used for the lower interface-identifier portion of
Optional prefix IAID	Non-standard identity-association ID for the tracked prefix, when r
Manual configuration	Allow manual DHCPv6 and Router Advertisement service configura
Gateway configuration	
Dynamic gateway policy	Create a gateway without a fixed next-hop address for supported t

Note

Identity Association offers similar functionality like *Track Interface (legacy)*, but without automatic ISC-DHCPv6 and Radvd configuration. It is intended for pure RA and DHCPv6 configuration using Dnsmasq or Kea/Radvd.

Mobile Networking



OPNsense supports 3G and 4G (LTE) cellular modems as failsafe or primary WAN interface. Both USB and (mini)PCIe cards are supported.

Supported Devices

While all devices supported by FreeBSD will likely function under OPNsense their configuration depends on a AT command string that can differ from device to device. To make thing easier some of these strings are part of a easy selectable profile.

Tested devices by the OPNsense team include:

- **Huaweu M909S-120** (device cuaUx.0) (Requires separate SIM card holder/adapter) [Tested: OPNsense 21.1]
- **Huawei ME909u-521** (device cuaUx.0)
- **Huawei E220** (device cuaUx.0)
- **Sierra Wireless MC7304** (device cuaUx.2) [as of OPNsense 16.7]

Note

If you have tested a cellular modem that is not on this list, but does work then please report it to the project so we can list it and inform others.

Configure Cellular modems

Setting up and configuring a cellular modem is easy, see: Configuring Cellular Modems

3G - 4G Cellular Failover

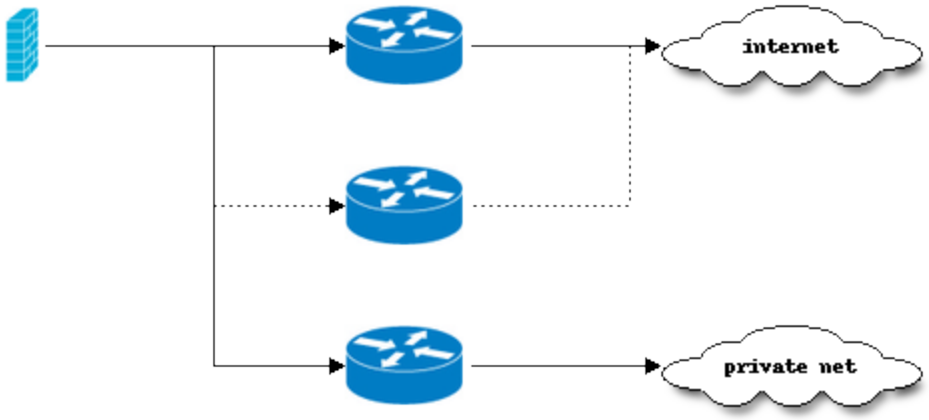
To setup Cellular Failover, just follow these two how-tos:

1. Configuring Cellular Modems
2. Multi WAN

Note

Treat the cellular connection the same as a normal WAN connection.

Gateways



Gateways define the possible routes that can be used to access other networks, such as the internet. All different paths that are available to your firewall can be managed from this page, which can be found at **System->Gateways->Configuration**.

You can either define these gateways yourself, or they can be provided automatically from dynamical configured interfaces (e.g. dhcp), in which case they won't have a predefined address.

When a gateway is generated automatically, you still have the ability to change its settings. Automatically generated gateways usually have names like `WAN_DHCP`.

In cases where you need to forward specific networks to a specific target, you can use static routes, which can be configured in **System->Routes->Configuration** and depend on the entries shown in the gateway page.

A specific kind of route is the `default` route, this is where all traffic is being send when no other static route is configured. There can only be one default at a time per ip protocol (ipv4, ipv6) in the system routing table.

Default gateways

Since there can only be one active default gateway, we need a method to figure out which one to use. For both ip protocols (ipv4, ipv6), this is determined equally.

Gateways have priorities, ranging from `1` [very important] to `255` [least important], automatically generated gateways will receive a low priority by default (which you can change manually).

Next there is a setting called `upstream`, which marks the gateway as favourable for default gateway selection, there can be more than one upstream configured at the same time.

When choosing a default, the algorithm will always sort `upstream` gateways higher (more attractive) and will use the priority next. If none of the gateways is explicitly chosen as upstream, the first non upstream is chosen.

In cases where gateway monitoring is configured, choosing a gateway also involves testing its current status and act accordingly when the monitored address is not reachable.

By default the system only chooses a (new) default gateway on startup or when an interface is connected or disconnected. In many cases you might want the default gateway also changed when the current gateway is not reachable anymore (via configured monitoring), in which case you can enable "Gateway switching" in **System->Settings->General**

Tip

In case you have multiple (dynamic) gateways, which should fall over in a specific order on failure, just set a **priority** and **upstream** flag. (e.g. favour fiber optics above 4g) Your choices should be reflected accordingly in the gateway grid (most important first).

Overview page

The overview page (**System->Gateways->Configuration**) shows all currently known gateways and their statuses in order of importance (most important on top). When a gateway is considered "default" it will show **(active)** behind the name.

Note

The **(active)** status shown in the list reflects the current calculated default, which might differ from the machine routing if gateway switching is not enabled. You can always check the current active default in **System->Routes->Status**

Tip

When debugging dynamic gateway issues, always check if your expected gateway is actually in the list. The system will only consider items shown here.

Settings

Below you will find the most important settings that are available for a gateway item.

Disabled	(temporary) disable this item
Name	Unique name for this gateway
Description	Optional description for this item
Interface	The interface this gateway is connected to
Address Family	IP family (v4 or v6)
IP address	Address of our gateway, empty/**dynamic** when dynamically gener
Upstream Gateway	Upstream gateway, consider this gateway as default gateway car
Far Gateway	Checkbox to allow the gateway to exist outside of the interface sul
Disable Gateway Monitoring	Disable monitoring (consider **online**)
Disable Host Route	Do not create a dedicated host route for this monitor.
Failover States	If this gateway goes down, force clients to reconnect over a differe
Failback States	If another gateway comes up with a higher priority than this gatev
Monitor IP	Alternative address to monitor, always make sure the address is a
Mark Gateway as Down	Consider this gateway as down, so it can't be considered as defau
Priority	Prioritizes this gateway, a lower value means more important.

Troubleshooting

Missing dynamic gateway

Most dynamic interface types write their current gateway into a file named `/tmp/[interface]_router` or `/tmp/[interface]_routerv6`. When the expected gateway is not listed in the page, there might be an issue with the interface type (for example the remote server isn't providing one at the moment).

Note

Some VPN types support sending traffic to the interface without knowing an intermediate gateway, to enable this check "Dynamic gateway policy" in the interface settings.

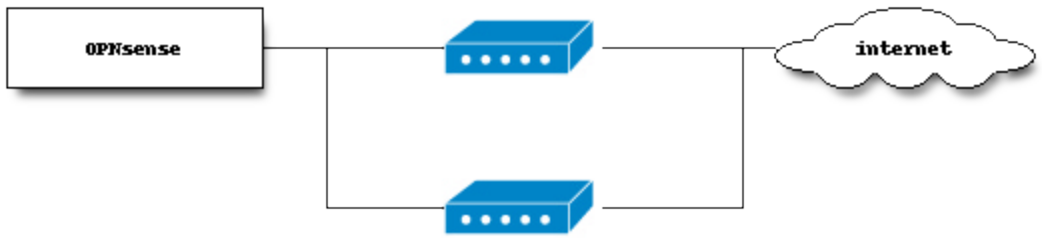
Gateway marked offline

When a gateway is marked offline, always check if the monitor address is reachable via the correct interface first. You can use **Interfaces->Diagnostics->Ping** to test connectivity. In case the monitor address is not in the same subnet as the interface, also check if there's a static route (**System->Routes->Status**) available which sends the requested traffic to the correct upstream gateway.

There should always be a `dpinger` process active, which you can check on the services page (**System->Diagnostics->Services**), finally if all seems to be running you can deep dive into the packets leaving the interface, if dpinger is active, there should be ICMP packets heading to your monitor address, which you can inspect using **Interfaces->Diagnostics->Packet Capture** .

Gateway groups / Multi WAN

Multi WAN scenarios are commonly used for failover or load balancing, but combinations are also possible with OPNsense.



The technology used to offer multiwan is called “policy based routing” or “source routing” and depends on the firewall functionality of OPNsense.

Note

Currently it’s not possible to use gateways without an address (Interface option “Dynamic gateway policy”) inside a group. This is due to the fact that the firewall requires an address of the right family (IPv4 / IPv6) to be present on the interface, which can not be guaranteed based on its configuration at the moment.

Terminology

When configuring gateway groups, there is a limited number of options and terms being used. Besides the name of the group, one can find the following terms on the page:

Gateway Priority	If a gateway is configured for a group, the ‘when’ part is divided into ‘tiers,’ wit
Trigger Level	When a gateway inside the tier is considered offline, either when its fully down
Pool Options	Usually left to default, but can influence stickiness for sources on a per group b

Roles

Using 'tiers', multiple scenarios can be constructed, by grouping gateways inside the same tier or choosing to move them to different ones. Below the most common scenarios.

WAN Failover

WAN failover automatically switches between WAN connections in case of connectivity loss (or high latency) of your primary ISP. As long as the connection is not good all traffic will be routed of the next available ISP/WAN connection and when connectivity is fully restored so will the routing switch back to the primary ISP.

WAN Load Balancing

Load balancing can be used to split the load between two (or more) ISPs. This enhances the total available bandwidth and/or lowers the load on each ISP.

The principle is simple: Each WAN connection (gateway) gets a portion of the traffic. The traffic can be divided equally or weighted.

Combining Balancing & Failover

It is also possible to combine Load Balancing with Failover in such scenarios you will have 2 or more WAN connections for Balancing purposes and 1 or more for Failover. OPNsense offers 5 tiers (Failover groups) each tier can hold multiple ISPs/WAN gateways.

Configuration

For a how to configure read:

- Multi WAN

DHCP

DHCP is used to automatically provide clients with an IP address (instead of clients having to set one themselves). DHCP is available for both IPv4 and IPv6 clients, referred to as DHCPv4 and DHCPv6, respectively.

Available Options

There are different DHCP servers/relays to choose from:

- Dnsmasq (default)
- KEA
- Dhcrelay
- ISC (EOL)

For DHCPv6 these services can offer Router Advertisements:

- radvd
- Dnsmasq

Dnsmasq DNS & DHCP

Dnsmasq Manual

Dnsmasq is a lightweight DNS, router advertisement and DHCP server. It is intended to provide coupled DNS and DHCP service to a LAN. Dnsmasq accepts DNS queries and either answers them from a small, local, cache or forwards them to a real, recursive, DNS server.

The dnsmasq DHCP server supports static address assignments and multiple networks. It automatically sends a sensible default set of DHCP options, and can be configured to send any desired set of DHCP options, including vendor-encapsulated options.

The dnsmasq DHCPv6 server provides the same set of features as the DHCPv4 server, and in addition, it includes router advertisements and a neat feature which allows naming for clients which use DHCPv4 and stateless autoconfiguration only for IPv6 configuration. There is support for doing address allocation (both DHCPv6 and RA) from subnets which are dynamically delegated via DHCPv6 prefix delegation.

Tip

Dnsmasq is the perfect DNS & DHCP server for small and medium sized setups (less than 1000 unique clients). It is the default for DHCPv4, DHCPv6 and Router Advertisements out of the box.

KEA DHCP

KEA Manual

KEA is a modern, modular, and high-performance DHCP server developed by ISC to succeed the legacy ISC DHCP server. It supports both DHCPv4 and DHCPv6 and is designed for scalable and high-availability environments.

KEA does not include DNS or router advertisement features, and is typically integrated with external services (such as *Unbound* DNS and *radvd* Router Advertisement Daemon) to provide full-stack DNS/DHCP/RA functionality.

Please note that there is no dynamic DNS lease registration functionality via Unbound implemented.

Tip

KEA is the perfect DHCP server for medium to large sized HA setups (more than 1000 unique clients) or environments requiring dynamic configuration via API.

ISC DHCP

ISC Manual

ISC DHCP was the DHCP server developed and maintained by the Internet Systems Consortium (ISC). It supported both DHCPv4 and DHCPv6 and was widely used across a broad range of systems and distributions for many years.

While it provided reliable service for static and dynamic address assignments, its monolithic architecture and limited extensibility led to challenges in modern, dynamic, or high-availability environments. Support for ISC DHCP has officially ended, and it has been fully replaced by KEA DHCP in most setups.

Attention

ISC DHCP is end-of-life and no longer receives updates or security patches. It is strongly recommended to migrate to KEA or Dnsmasq.

DHCRelay

DHCrelay Manual

DHCP relaying is the forwarding of DHCP requests received on one interface to a DHCP server on another network. This is useful when the DHCP server is not on the same broadcast domain as the client.

The *dhcrelay* service supports both DHCPv4 and DHCPv6 and can forward requests to multiple upstream servers. It is lightweight and suitable for setups where centralized DHCP servers serve multiple network segments.

Note

DHCrelay binds to port 67 like other available DHCP servers. To run multiple servers side by side, use strict interface binding if available.

Reservations

ISC, KEA and Dnsmasq offer the possibility to reserve an IP address for a specific client. This is useful when a client needs to have the same IP address every time it connects to the network. All services also offer the ability to define reservations inside and outside of the assigned pool of dynamic IP addresses.

For **Dnsmasq**, you should define reservations **inside of the pool**. The IP address will be completely reserved inside the dynamic range, meaning the reserved IP will not be offered to dynamic clients.

For **ISC and KEA**, you should only define reservations **outside of the pool**. Unless you can guarantee that this client is online at all times when the reservation is in the dynamic range, the DHCP server is free to offer this IP address to a different client when the first client goes offline.

Unbound DNS

Unbound is a validating, recursive, caching DNS resolver. It is designed to be fast and lean and incorporates modern features based on open standards.

Since OPNsense 17.7 it has been our standard DNS service, which on a new install is enabled by default.

General settings

Warning

Below table contains the options to manually set **listening** and **outbound** interfaces, the recommended setting for both is `"All"` for good reasons. Unless you absolutely know what you are doing, best keep these settings default as misuse often causes startup issues.

Below you will find the most relevant settings from the **General** menu section.

Enable	Enable our DNS resolver
Listen Port	Port to listen on, when blank, the default (53) is used.
Network Interfaces	Interface IP addresses used for responding to queries from clie
DNSSEC	Enable DNSSEC to use digital signatures to validate results fro
DNS64	Enable DNS64 so IPv6-only clients can reach IPv4-only servers
AAAA-only mode	If this option is set, Unbound will remove all A records from the
Register ISC DHCP4 Leases	**IPv4 only** If this option is set, then machines that specify their h The source of this data is **client-hostname** in the dhcpd.leases fi
DHCP Domain Override	When the above registrations shouldn't use the same domain r
Register DHCP Static Mappings	Register static dhcpd entries so clients can resolve them. Suppo
No IPv6 Link-local aaddresses	Do not register link local addresses for IPv6. This will prevent th
System A/AAAA records	If this option is set, then no A/AAAA records for the configured I

TXT Comment Support	Register descriptions as comments for dhcp static host entries.
Force SafeSearch	Force the usage of SafeSearch on Google, DuckDuckGo, Bing, C
Outgoing Network Interfaces	Utilize different network interfaces that Unbound will use to ser
Local Zone Type	The local zone type used for the system domain. Type descripti

Note

In order for the client to query unbound, there need to be an ACL assigned in **Services ▸ Unbound DNS ▸ Access Lists**. The configured interfaces should gain an ACL automatically. If the client address is not in any of the predefined networks, please add one manually.

Overrides

Within the overrides section you can create separate host definition entries and specify if queries for a specific domain should be forwarded to a predefined server.

Host override settings

Host overrides can be used to change DNS results from client queries or to add custom DNS records. PTR records are also generated under the hood to support reverse DNS lookups. These are generated in the following way:

- If **System A/AAAA records** in General settings is unchecked, a PTR record is created for the primary interface.
- Each host override entry **that does not include a wildcard for a host**, is assigned a PTR record.
- If a host override entry **includes a wildcard for a host**, the first defined alias is assigned a PTR record.
- Every other alias does not get a PTR record.

Host	Name of the host, without domain part. Use "*" to create a wildcard entry.
Domain	Domain of the host (such as example.com)
Type	Record type, A or AAA (IPv4 or IPv6 address), MX to define a mail exchange
IP	Address of the host
Description	User readable description, only for informational purposes

Aliases	Copies of the above data for different hosts

Aliases

You may create alternative names for a Host. E.g. when having a webserver with several virtual hosts you create a Host override entry with the IP and name for the webserver and an alias name for every virtual host on this webserver.

You have to select the host in the top list and it will the show you the assigned aliases in the bottom list.

Domain override settings

Important

Domain overrides has been superseded by Query Forwarding. Query forwarding also allows you to forward every single request.

Advanced

Although the default settings should be reasonable for most setups, some need more tuning or require specific options set. Some of these settings are enabled and given a default value by Unbound, refer to unbound.conf(5) for the defaults.

Hide Identity	If enabled, id.server and hostname.bind queries are ref
Hide Version	If enabled version.server and version.bind queries are r
Prefetch Support	Message cache elements are prefetched before they e
Prefetch DNS Key Support	DNSKEY's are fetched earlier in the validation process
Harden DNSSEC data	DNSSEC data is required for trust-anchored zones. If su
Serve expired responses	Serve expired responses from the cache with a TTL of C
Expired Record Reply TTL Value	TTL value to use when replying with expired data. If "Cl
TTL for Expired Responses	Limits the serving of expired responses to the configure
Reset Expired Record TTL	Set the TTL of expired records to the "TTL for Expired R
Client Expired Response Timeout	Time in milliseconds before replying to the client with e

Strict QNAME Minimisation	Send minimum amount of information to upstream ser
Extended Statistics	If enabled, extended statistics are printed to syslog.
Log Queries	If enabled, prints one line per query to the log, with the
Log Replies	If enabled, prints one line per reply to the log, with the l
Tag Queries and Replies	If enabled, prints the word 'query: ' and 'reply: ' with log
Log level verbosity	Select the log verbosity. Level 0 means no verbosity, or
Private Domains	List of domains to mark as private. These domains and
Rebind Protection networks	These are addresses on your private network, and are r
Insecure Domains	List of domains to mark as insecure. DNSSEC chain of t
Message Cache Size	Size of the message cache. The message cache stores l
RRset Cache Size	Size of the RRset cache. Contains the actual RR data. V
Outgoing TCP Buffers	The number of outgoing TCP buffers to allocate per thr
Incoming TCP Buffers	The number of incoming TCP buffers to allocate per thr
Number of queries per thread	The number of queries that every thread will service sir
Outgoing Range	The number of ports to open. This number of file descri
Jostle Timeout	This timeout is used for when the server is very busy. Se
Maximum TTL for RRsets and messages	Configure a maximum Time to live in seconds for RRset
Minimum TTL for RRsets and messages	Configure a minimum Time to live in seconds for RRset
TTL for Host cache entries	Time to live in seconds for entries in the host cache. The
Keep probing down hosts	Keep probing hosts that are down in the infrastructure
Number of Hosts to cache	Number of hosts for which information is cached.
Unwanted Reply Threshold	If enabled, a total number of unwanted replies is kept t

Access Lists

Access lists define which clients may query our dns resolver. Records for the assigned interfaces will be automatically created and are shown in the overview. You can also define custom policies, which apply an action to predefined networks.

Note

The action can be as defined in the list below. The most specific netblock match is used, if none match deny is used. The order of the access-control statements therefore does not matter.

Actions

Deny	This action stops queries from hosts within the defined networks.
Refuse	This action also stops queries from hosts within the defined networks, but sen
Allow	This action allows queries from hosts within the defined networks.
Allow Snoop	This action allows recursive and nonrecursive access from hosts within the def
Deny Non-local	Allow only authoritative local-data queries from hosts within the defined netv
Refuse Non-local	Allow only authoritative local-data queries from hosts within the defined netv

Blocklists

Enable integrated dns blocklisting using one of the predefined sources or custom locations.

Multiple policies can be defined, each separated by one or more source nets. This means you can use blocklists or specific (wildcard) domains on specific networks, allowing more fine-grained control over your setup. The algorithm selects the most specific subnet when domains overlap across subnet sizes. You can use the tester as described below to find such overlaps, however, as a best practice it's advisable to keep source nets separate.

Enable	Enable blocklists
Type of DNSBL	Predefined external sources
URLs of Blocklists	Additional http[s] location to download blocklists from, only plain text files
Allowlist Domains	When a blocklist item contains a pattern defined in this list it will be omitte
Blocklist Domains	List of domains to explicitly block. Regular expressions are not supported. F
Wildcard Domains	List of wildcard domains to blocklist. All subdomains of the given domain w
Source Net(s)	Source networks to apply policy on. Examples are 192.168.1.0/24 or 192.168.1.
Cache TTL	TTL for the blocklists cache. Remote blocklists don't usually update more of

Destination Address	Specify an IP address to return when DNS records are blocked. Can be use
Return NXDOMAIN	Instead of returning the "Destination Address", return the DNS return code

Note

Applying the blocklist settings will not restart Unbound, rather it will signal to Unbound to dynamically process the blocklists as soon as they're downloaded. There may be up to a minute of delay before Unbound has loaded everything. During this time Unbound will still be just as responsive.

Blocklist Tester

The blocklists feature contains a tester that allows you to check whether a given domain matches a policy you have defined in the *Blocklists* tab. This tester is fully local, therefore this tester is not capable of matching domains that resolve into CNAMEs that may or may not be in the blocklists you have defined. In this scenario you can use **Interfaces -> Diagnostics -> DNS lookup**, or use the following command from the command line:

```
drill -b <source ip> <domain> <record type>
```

In the above example the source IP address must exist on the firewall.

Predefined sources

When any of the DNSBL types are used, the content will be fetched directly from its original source, to get a better understanding of the source of the lists we compiled the list below containing references to the list maintainers.

Abuse.ch - ThreatFox IOC database	https://threatfox.abuse.ch/
AdAway List	https://adaway.org/hosts.txt
AdGuard List	https://v.firebog.net/hosts/AdguardDNS.txt
OISD - Domain Blocklist Ads*	https://small.oisd.nl/domainswild
OISD - Domain Blocklist Big*	https://big.oisd.nl/domainswild
OISD - Domain Blocklist NSFW*	https://nsfw.oisd.nl/domainswild

Blocklist.site	https://github.com/blocklistproject/Lists
EasyList	https://v.firebog.net/hosts/Easylist.txt
Easyprivacy	https://v.firebog.net/hosts/Easyprivacy.txt
YoYo List	https://pgl.yoyo.org/adservers/
hagezi - [multiple lists]	https://github.com/hagezi/dns-blocklists

Note

The OISD lists are *wildcard* lists. Meaning that they will block all subdomains of the listed domains. For more information, refer to OISD. This keeps the list small and manageable, but are more effective than regular lists.

Note

In order to automatically update the lists on timed intervals you need to add a cron task, just go to **System -> Settings ->Cron** and a new task for a command called "Update Unbound DNSBLs".

Usually once a day is a good enough interval for these type of tasks.

Query Forwarding

The Query Forwarding section allows for entering arbitrary nameservers to forward queries to. It is assumed that the nameservers entered here are capable of handling further recursion for any query. In this section you are able to specify nameservers to forward to for specific domains queried by clients, catch all domains and specify nondefault ports.

Use System Nameservers	The configured system nameservers will be used to forward queries t

Warning

Do not use the system nameservers option if you have a multi-WAN setup and have Unbound running alongside multiple DNS servers configured in General with separate gateways assigned to them. Unbound will use the locally created routes to reach the system nameservers, which will not work when the gateway is down.

Note

Keep in mind that if the "Use System Nameservers" checkbox is checked, the system nameservers will be preferred over any **catch-all entry** in **both** Query Forwarding and DNS-over-TLS, this means that entries with a specific domain will still be forwarded to the specified nameserver.

Enabled	Enable query forwarding for this domain.
Domain	Domain of the host. All queries for this domain will be forwarded to the nameserver sp
Server IP	Address of the DNS server to be used for recursive resolution.
Port	Specify the port used by the DNS server. Default is port 53. Useful when configuring e.g

Warning

Be careful enabling "DNS Query Forwarding" in combination with **DNSSEC**, no DNSSEC validation will be performed for forwards with a specific domain, as the upstream server might be a local controller. If forwarding everything and the upstream server doesn't support DNSSEC, its answers will not reach the client as no DNSSEC validation could be performed.

DNS over TLS

DNS over TLS uses the same logic as Query Forwarding, except it uses TLS for transport.

Note

Please be aware of interactions between Query Forwarding and DNS over TLS. Since the same principle as Query Forwarding applies, a **catch-all entry** specified in both sections will be considered a duplicate zone. In our case DNS over TLS will be preferred.

Enabled	Enable DNS over TLS for this domain.
Domain	Domain of the host. All queries for this domain will be forwarded to the nameserver sp
Server IP	Address of the DNS server to be used for recursive resolution.

Port	Specify the port used by the DNS server. Always enter port 853 here unless there is a g
Verify CN	The name to use for certificate verification, e.g. "445b9e.dns.nextdns.io". Used by Unbo

Tip

To ensure a validated environment, it is a good idea to block all outbound DNS traffic on port 53 using a firewall rule when using DNS over TLS. Should clients query other nameservers directly themselves, a NAT redirect rule to 127.0.0.1:53 (the local Unbound service) can be used to force these requests over TLS.

Public Resolvers

Hosted by	Server IP	Server Port	Verify CN
Cloudflare	1.1.1.1	853	cloudflare-dns.com
	1.0.0.1		
	2606:4700:4700::1111		
	2606:4700:4700::1001		
Google	8.8.8.8	853	dns.google
	8.8.4.4		
	2001:4860:4860::8888		
	2001:4860:4860::8844		
Quad9	9.9.9.9	853	dns.quad9.net
	149.112.112.112		
	2620:fe::fe		
	2620:fe::9		

Statistics

The statistics page provides some insights into the running server, such as the number of queries executed, cache usage and uptime.

Advanced Configurations

Some installations require configuration settings that are not accessible in the UI. To support these, individual configuration files with a `.conf` extension can be put into the `/usr/local/etc/unbound.opnsense.d` directory. These files will be automatically included by the UI generated configuration. Multiple configuration files can be placed there. But note that

- As it cannot be predicted in which clause the configuration currently takes place, you must prefix the configuration with the required clause. For the concept of "clause" see the `unbound.conf(5)` documentation.
- The wildcard include processing in Unbound is based on `glob(7)`. So the order in which the files are included is in ascending ASCII order.
- Name collisions with plugin code, which use this extension point e. g. `dnsbl.conf`, may occur. So be sure to use a unique filename.
- It is a good idea to check the complete configuration via:

  ```
  # check if the resulting configuration is valid
  configctl unbound check
  ```

  This will report errors that prevent Unbound from starting and also list warnings that may give hints as to why a particular configuration is not working or how it could be improved.

This is a sample configuration file to add an option in the server clause:

```
server:
  private-domain: xip.io
```

Note

As a more permanent solution the template system ("Using Templates") can be used to automatically generate these files.

To get the same effect as placing the file in the sample above directly in `/usr/local/etc/unbound.opnsense.d` follow these steps:

1. Create a `+TARGETS` file in `/usr/local/opnsense/service/templates/sampleuser/Unbound`:

```
sampleuser_additional_options.conf:/usr/local/etc/unbound.opnsense.d/sampleuser_add
itional_options.conf
```

2. Place the template file as `sampleuser_additional_options.conf` in the same directory:

```
server:
  private-domain: xip.io
```

3. Test the template generation by issuing the following command:

```
# generate template
configctl template reload sampleuser/Unbound
```

4. Check the output in the target directory:

```
# show generated file
cat /usr/local/etc/unbound.opnsense.d/sampleuser_additional_options.conf
# check if configuration is valid
configctl unbound check
```

Warning

It is the sole responsibility of the administrator which places a file in the extension directory to ensure that the configuration is valid.

Note

This method replaces the `Custom options` settings in the General page of the Unbound configuration, which was removed in version 21.7.

Dynamic DNS

In order to update DNS records when the firewall's IP address changes, use a dynamic DNS service provider.

Our *os-ddclient* plugin offers support for various dynamic DNS services using either the ddclient software or our native backend.

Prerequisites

Before installing and using this plugin, make sure to register an account with one of the supported DNS providers.

Installation

Installation of this plugin is rather easy, go to **System ▸ Firmware ▸ Plugins** and search for **os-ddclient**, use the [+] button to install it.

Next go to **Services ▸ Dynamic DNS ▸ Settings** to configure one or more Dynamic DNS services.

General settings

The general settings tab allows you to customize the services settings which will affect all dynamic DNS accounts.

Option	Description
Enable	Enable or disable the service
Verbose	Enable or disable detailed log messages for the service
Allow IPv6	Enable or disable retrieving and updating IPv6 addresses
Interval	How often (in seconds) the service checks if the appliance's IP addresses have chang
Backend	The system the Select the backend to use, either "ddclient" or "native" (see below)

Note

With `ddclient` developments sunsetting [*] we decided to offer an alternative written in Python. Selecting the native backend replaces the employed implementation. If your service is supported, we do advise to try out the new native backend which also offers support for custom HTTP requests.

Accounts

The accounts tab allows you to configure one or more dynamic DNS providers which the service will update using an API call over HTTP(S).

Option	Description
Enabled	Enable or disable the account (allows turning entries off without removing
Description	A descriptive name for the account
Service	The provider of your Dynamic DNS Service
resourceId	A pointer to the service to be updated, currently only relevant for Azure
Username	Login or user name to authenticate with, left empty when using token-base
Password	Password, API key or security token to authenticate with
Zone	Typically the second-level domain name to update. For example: *example.*
Hostname(s)	A list of fully-qualified domain names to update via the selected service, se
Check ip method	The IP checking service to get the appliance's current IP address from
Interface to monitor	The interface to connect to the IP checking service from, or if the "Check ip
Dynamic ipv6 host	Optionally overwrite the interface identifier of the ipv6 address with the giv
Check ip timeout	How long to wait in seconds for the IP checking service to respond with an
Force SSL	Enable connecting to IP checking and updating services over HTTPS, thoug

Note

To update a fully-qualified domain name's `A` (IPv4) record and `AAAA` (IPv6) record, you must: * Add one account for IPv4 with the "Check ip method" set to an IPv4 service * Add one account for IPv6 with the "Check ip method" set to an IPv6 service * On the *General Settings* tab, enable "Allow IPv6"

Provider-specific configuration

Cloudflare

For accounts with Cloudflare as provider, there is an additional option **Zone**, which should be set as the name of the zone containing the host to be updated, not its zone ID.

Cloudflare accepts authorization with the global token with the options

Option	Value
Username	The email of the Cloudflare account.
Password	Global API Key.

Using an API token is recommended for security reasons, with `Permissions` **Zone ‣ DNS ‣ Edit** and `Zone Resources` **Include ‣ Specific zone ‣ zone with the host**, and the account options

Option	Value
Username	token
Password	API token.

Netcup DNS

Netcup is a German hosting provider who offers an API for DNS manipulation:

- Wiki: https://www.netcup-wiki.de/wiki/DNS_API
- Technical documentation: https://ccp.netcup.net/run/webservice/servers/endpoint.php

Option	Value
Username	customer number
Password	APIPassword|APIKey, both fields need to be concatenated using a pipe (`|`) symbol

Mythic Beasts

Mythic Beasts is a UK based hosting provider who offers an API for DNS manipulation:

- Wiki: https://www.mythic-beasts.com/support/API/DNSv2/dynamic-DNS
- Technical documentation: https://www.mythic-beasts.com/support/API/DNSv2

Option	Value
Username	Key ID
Password	Secret

Network Time

OPNsense ships with a standard NTPd server, which synchronizes time with upstream servers and provides time to connected clients.

A newly installed firewall comes with NTP enabled on all interfaces (firewall blocks all non LAN access in this case), forwarding queries to one of the `X.opnsense.pool.ntp.org` upstreams (`X` is any of 0,1,2,3).

General settings

In most cases the default setup is ready to use, below you will find some of the general options which can be configured.

Attention

NTPd syncs the local clock to remote NTP servers and listens for incoming client queries using the selected interfaces. The interface selection must therefore include a WAN type interface so that normal routing to the internet can take place. Only selecting a LAN type interface will likely work due to Source NAT rules, however, if CARP is configured, it is possible that such NAT rules will translate to the address of the CARP VIP, causing asymmetric routing for secondary nodes.

Unless you are sure that you want to use a specific interface, it is recommended to keep the default of listening on all interfaces and use the firewall rules to constrain access to the NTP service. See Listen interfaces for more information.

Interface(s)	Interfaces to listen on. This interface selection is also used for outgoing queri

Time servers	Servers to use, comes with two toggles: • **Prefer** Marks the server as preferred. • **Do not use** Marks the server as unused, except for display purposes. The serv
Orphan mode	Orphan mode allows the system clock to be used when no other clocks are a
NTP graphs	Enable RRD graphs of NTP statistics, which can be viewed in **Reporting ▸ Healt**
Syslog logging	Extend logging with peer and/or system messages
Statistics logging	Enable statistical logging in `/var/log/ntp`, doesn't come with a user interfac
Access restrictions	Within the access restriction row, you can set various options which limit the
Leap seconds	You can manually supply ntpd with a leap seconds file, more detailed info on

Note

NTPs is disabled if no Time servers are configured. There is no separate enable/disable toggle.

GPS

If you own a gps receiver, which supports NMEA, you can use it as a reference clock and configure it in this section. For some brands settings are preconfigured, you can also use custom settings.

PPS

If your GPS receiver supports PPS (Pulse Per Second) output or you have a separate PPS signal available, you can configure the serial port to use along with some other settings here.

Status

The status page can be used to query the health of the upstream time sources, it shows the following characteristics.

Status	Human readable status of the upstream
Server	Upstream source
Ref ID	Source IP address or kiss code:
Stratum	NTP uses a hierarchical, semi-layered system of time sources (wikipedia) This field shov
Type	Type of time source, one of: • u: unicast or manycast client • b: broadcast or multicast client • p: pool source • l: local (reference clock) • s: symmetric (peer) • A: manycast server • B: broadcast server • M: multicast server
When	time in seconds, minutes, hours, or days since the last packet was received, or '-' if a pac
Poll	poll interval in seconds
Reach	reach shift register (octal), when set to `377` all 8 bits are set and thus no responses hav means the last four responses where received and two where lost within the last 8 tries
Delay	roundtrip delay in milliseconds
Offset	offset time in milliseconds of the server relative to this host
Jitter	indicates the difference, in milliseconds, between two samples

Wireless

Although wireless networks are supported in OPNsense, result may vary.

In our experience most companies use separate access points to facilitate WiFi, for reasons as supported technology (nowadays most devices expect wireless-ac, which isn't supported), stable hardware and often the location where the firewall is installed plays an important role (signal strength issues).

Our documentation on the subject is limited, contributions will be shown below.

- Interfaces: Wireless Networks (INTERNAL)

Rules

Index

- Rules
  - Overview
  - The basics
    - States
    - Action
    - Processing order
    - Rule sequence
    - Direction
  - Implementations
    - Firewall Rules
      - User Interface
        - Interface filter
        - Categories filter
        - Tree button
        - Inspect button
      - Settings
      - Divert-to
    - Legacy Firewall Rules
      - User Interface
      - Settings
  - Troubleshooting

Overview

OPNsense contains a stateful packet filter, which can be used to restrict or allow traffic from and/or to specific networks as well as influence how traffic should be forwarded (see also policy based routing in "Multi WAN").

The rules section shows all policies that apply on your network, grouped by interface.

There are two firewall rule implementations:

- **Firewall Rules**: the default modern MVC implementation with API support and improved rule management.
- **Firewall Rules [legacy]**: static PHP pages supplied by the optional `os-firewall-legacy` plugin.

Tip

Existing legacy rules continue to work without the plugin and can be migrated using **Firewall ‣ Migration Assistant**.

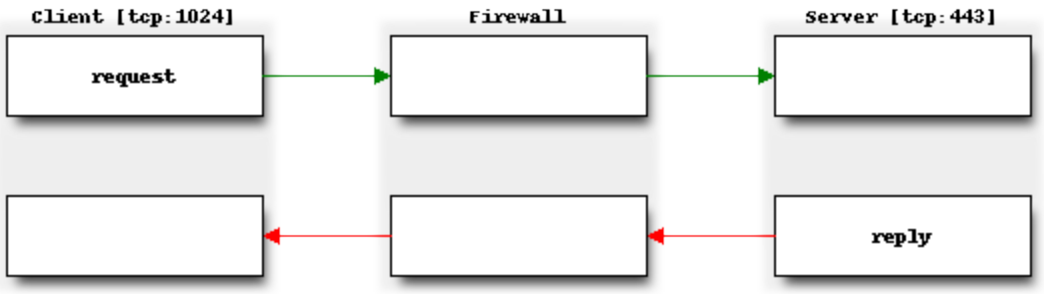
The basics

Before creating rules, it's good to know about some basics which apply to all rules.

States

By default rules are set to stateful (you can change this, but it has consequences), which means that the state of a connection is saved into a local dictionary which will be resolved when the next packet comes in. The consequence of this is that when a state exists, the firewall doesn't need to process all its rules again to determine the action to apply, which has huge performance advantages.

Another advantage of stateful packet filtering is that you only need to allow traffic in one direction to automatically allow related packets for the same flow back in. Below diagram shows a tcp connection from a client to a server for https traffic, when not using stateful rules, both the client should be permitted to send traffic to the server at port 443 as the server back to the client (usually a port >=1024).



The use of states can also improve security particularly in case of tcp type traffic, since packet sequence numbers and timestamps are also checked in order to pass traffic, it's much harder to spoof traffic.

Note

When changing rules, sometimes its necessary to reset states to assure the new policies are used for existing traffic. You can do this in **Firewall ▸ Diagnostics ▸ States**.

Note

In order to keep states, the system need to reserve memory. By default 10% of the system memory is reserved for states, this can be configured in **Firewall ▸ Settings ▸ Firewall Maximum States**. (The help text shows the default number of states on your platform)

States can also be quite convenient to find the active top users on your firewall at any time, we added an easy to use "session" browser for this purpose. You can find it under **Firewall ▸ Diagnostics ▸ Sessions**.

Tip

States also play an important rule into protecting services against (distributed) denial of service attacks (DDOS). Relevant topics available in our documentation are "synproxy" states, connection limits and syncookies

Action

Rules can be set to three different action types:

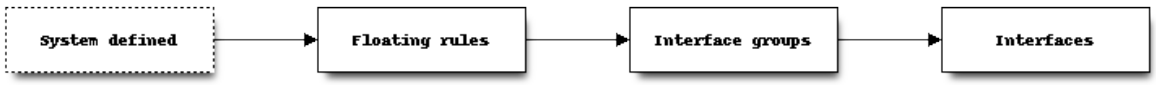
- Pass –> allow traffic
- Block –> deny traffic and don't let the client know it has been dropped (which is usually advisable for untrusted networks)
- Reject –> deny traffic and let the client know about it. (only tcp and udp support rejecting packets, which in case of TCP means a `RST` is returned, for UDP `ICMP UNREACHABLE` is returned).

For internal networks it can be practical to use reject, so the client does not have to wait for a time-out when access is not allowed. When receiving packets from untrusted networks, you usually don't want to communicate back if traffic is not allowed.

Processing order

Firewall rules are processed in sequence per section, first evaluating the **Floating** rules section followed by all rules which belong to **interface groups** and finally all **interface** rules.

Internal (automatic) rules are usually registered first.



Rules can either be set to `quick` or not set to quick, the default is to use quick. When set to quick, the rule is handled on "first match" basis, which means that the first rule matching the packet will take precedence over rules following in sequence.

When `quick` is not set, last match wins. This can be useful for rules which define standard behaviour. Our default deny rule uses this property for example (if no rule applies, drop traffic).

Note

Internally rules are registered using a priority, floating uses `200000`, groups use `300000` and interface rules land on `400000` combined with the order in which they appear. Automatic rules are usually registered at a higher priority (lower number).

Warning

NAT rules are always processed before filter rules! So for example, if you define a NAT : Destination NAT (Port Forwarding) rules *without a associated rule*, i.e. **Filter rule association** set to **Pass**, this has the consequence, that no other rules will apply!

Tip

The interface should show all rules that are used, when in doubt, you can always inspect the raw output of the ruleset in `/tmp/rules.debug`

When the optional `os-firewall-legacy` plugin is installed, legacy rules and Firewall Rules can exist side by side. The following additional processing-order considerations apply.

If a filter rule has:

- a single interface defined, it is an **Interface Rule**
- a group interface defined, it is a **Group Rule**
- any number of interfaces or one inverted interface defined, it is a **Floating Rule**

Processing order:

1. System defined rules at the beginning of the ruleset
2. **Firewall ▸ Rules** and **Firewall ▸ Rules [legacy]** floating rules
3. **Firewall ▸ Rules** and **Firewall ▸ Rules [legacy]** group rules
4. **Firewall ▸ Rules** single-interface rules
5. **Firewall ▸ Rules [legacy]** single-interface rules
6. System defined rules at the end of the ruleset

Rule sequence

The sequence in which the rules are displayed and processed can be customized per section:

- Select a rule with the checkbox on the left side of the rule.
- Use the arrow button in the action menu on the right side of a rule in order to move selected rules before the rule where the action button is pressed.

While rules in **Firewall ▸ Rules [legacy]** are processed implicitly by the order in which they appear in the configuration file, **Firewall ▸ Rules** uses an explicit **Sort order**.

A **Sort order** will have this structure: `200000.0000250`:

- `200000` The **Priority group**, defines the hierarchy the rule belongs to. It can be influenced by the interface in a rule.
- `.0000250` The **Sequence**, it defines the exact spot of the rule in context of the interface hierarchy. It can be influenced with the *Sequence* field inside a rule, or with the *Move rule before this rule* button.

As example, we create a few different rules:

- `200000.0000100` (Floating rule)
- `200000.0000200` (Floating rule)
- `300000.0000050` (Group rule)
- `300000.0000060` (Group rule)
- `400000.0002000` (Interface rule)
- `400000.0003100` (Interface rule)

When applying the filter, the rules will be processed in this **Sort order**. A lower sequence number inside the priority group means the priority is higher.

Note

The *Sequence* does not have to be unique, multiple rules can share the same number. Though what this means is that the filter is not populated as strictly in order as if all rules have a unique sequence.

Tip

When adding a new rule, the *Sequence* will be automatically populated with a unique number. You can either change this number manually to adjust the exact position of the rule, or use the *Move rule before this rule* button.

Direction



Traffic can be matched on `in[coming]` or `out[going]` direction, our default is to filter on incoming direction. In which case you would set the policy on the interface where the traffic originates from.

For example, if you want to allow `https` traffic coming from any host on the internet, you would usually set a policy on the WAN interface allowing port `443` to the host in question.

Note

Traffic leaving the firewall is accepted by default (using a non-quick rule), when **Disable force gateway** in **Firewall ▸ Settings ▸ Advanced** is not checked, the connected gateway would be enforced as well.

Implementations

Firewall Rules

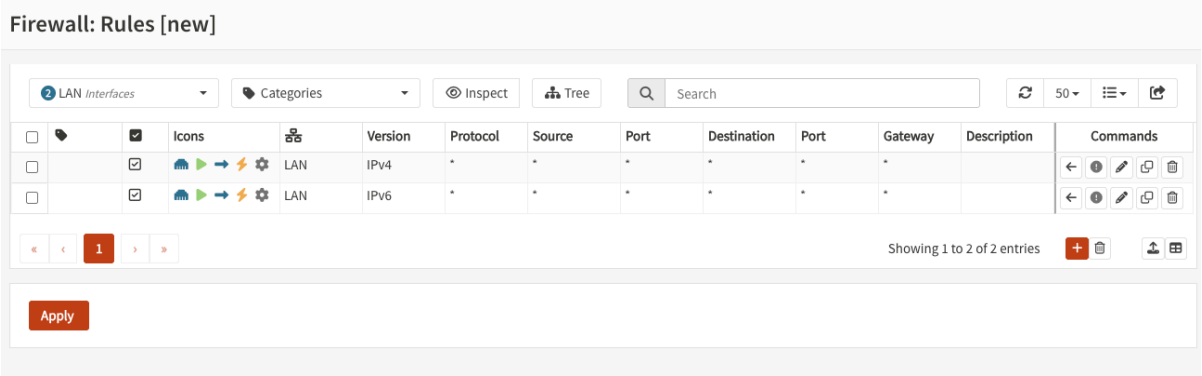
Firewall Rules provides API support and a reworked GUI with advanced search, category folders, improved performance, and better visibility.

Open Firewall Rules from **Firewall ▸ Rules**.

User Interface

Our overview shows all the rules that apply to the selected interface, group or floating section. For every rule, some details are provided and when applicable you can perform actions such as move, edit, copy, delete.

API access is described in more detail in the firewall API reference manual.



Interface filter

Choose an interface to filter the current view. Floating, group and single interfaces can be selected.

If you choose the “LAN” interface, you will be presented with all floating, group and single interface rules that influence packet decisions of the LAN interface.

If you create a new rule while having an interface selected, it will be automatically added to dialog.

Categories filter

Choose one or multiple categories to filter the current view. This combines with the selection of the interface filter.

Categories can be created in **Firewall ▸ Categories** and can enable grouping different logic constructs.

If you create a category for mailservers and tag rules with it, you can simply filter for this tag and only see your mailservers. As with the interface filter, selecting one or multiple tags will add them automatically to a new rule.

Tree button

Group all firewall rules in a tree, based on their category. The groups act like folders, but always honor the sort order and sequence. The group folders are not saved inside the configuration, they are an alternative view based on categories.

Whenever a rule in sequence changes category, a new folder copying the category of the first rule will be created. To move similar rules into the same folder, change their sequence and category. But keep in mind that the sequence of rules will always be the same as without the tree view.

Inspect button

The inspect button will reveal all system defined (automatic) firewall rules and show rule statistics. It can be enabled at any time to get a complete view of the current active ruleset.

While in inspect mode the search can find IP addresses inside aliases.

Rule statistics are cached. To pull the latest statistics, press the refresh button in the statistics column.

Settings

Organisation | Interface | Filter | Stateful firewall | Rate limiting | Traffic shaping [experimental] | Source Routing | Priority | Internal tagging | Audit

Option	Description
Enabled	Enable this rule
Sort order	The order in which rules are being processed.
Sequence	The order in which rules are being processed. Please note that this is not
Categories	For grouping purposes you may select multiple groups here to organize it
No XMLRPC Sync	Exclude this item from the HA synchronization process. An already existir
Description	You may enter a description here for your reference (not parsed).

Divert-to

With divert we can integrate additional policy constructs into firewall rules. A rule with "divert-to" can send all packets to a divert socket, on which a service like Intrusion Prevention System can listen.

This can provide more fine grained control than intercepting all traffic for inspection. It can improve performance because large flows that do not necessarily need inspection can be excluded.

As an example, go to **Services ‣ Intrusion Detection ‣ Administration** and set the "Capture mode" to "Divert (IPS)".

Afterwards, create a firewall rule that matches the traffic that should be inspected and set the "Divert-to" settings in that rule. Matching packets will be diverted to the socket the intrusion detection listens on. After making a decision, the packet will then be forwarded or dropped.

Attention

Keep in mind that when the service that listens on the divert socket is stopped, the firewall rule will drop all matching packets.

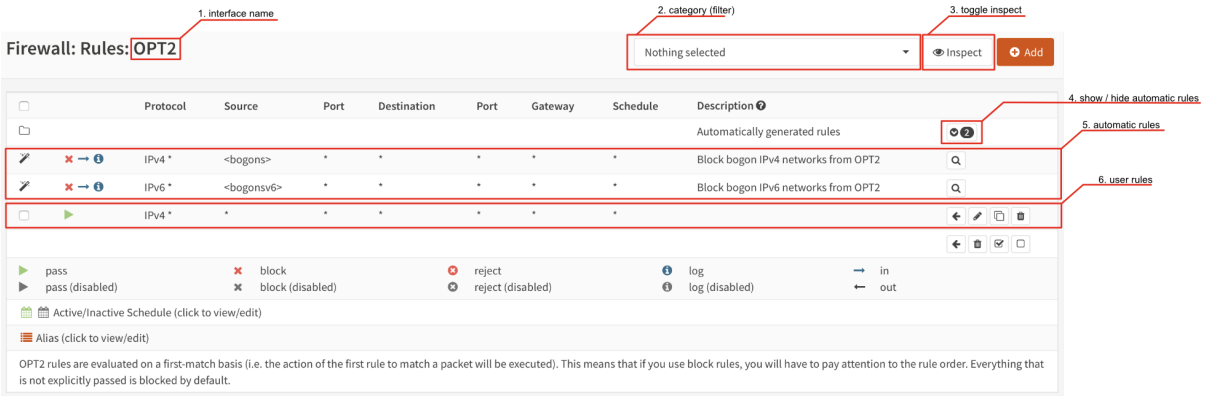
Legacy Firewall Rules

Legacy Firewall Rules are static PHP pages supplied by the optional `os-firewall-legacy` plugin. They have no API support and are not installed by default. Install the plugin only when legacy rule management is required; existing legacy rules continue to work without it and can be migrated using **Firewall ▸ Migration Assistant**.

When installed, the plugin adds the legacy firewall rule pages and the legacy **Firewall ▸ NAT ▸ Outbound** page.

User Interface

Our overview shows all the rules that apply to the selected interface (group) or floating section. For every rule some details are provided and when applicable you can perform actions, such as move, edit, copy, delete.



Below you will find some highlights about this screen.

1. **Interface name**

   The name of the interface is part of the normal menu breadcrumb

2. **Category**

   If categories are used in the rules, you can select which one you will show here.

3. **Toggle inspection**

You can toggle between inspection and rule view here, when in inspection mode, statistics of the rule are shown. (such as packet counters, number of active states, ...)

4. **Show / hide automatic rules**

   Some rules are automatically generated, you can toggle here to show the details. If a magnifying glass is shown you can also browse to its origin (The setting controlling this rule).

5. **Automatic rules**

   The contents of the automatic rules

6. **User rules**

   All user defined rules

Settings

Traffic that is flowing through your firewall can be allowed or denied using rules, which define policies. This section of the documentation describe the different settings, grouped by usage.

Organisation | Basic settings | Traffic shaping (QoS) | Less commonly used

High Availability | Schedule | Policy based routing | Connection limits

Advanced

Some settings help to identify rules, without influencing traffic flow.

Option	Description
Category	The category this rule belongs to, can be used as a filter in the overview
Description	Descriptive text

Troubleshooting

While building your ruleset things can go wrong, it's always good to know where to look for signs of an issue. One of the most common mistakes is traffic doesn't match the rule and/or the order of the rule doesn't make sense for whatever reason.

With the use of the "inspect" button, one can easily see if a rule is being evaluated and traffic did pass using this rule. It's also possible to jump directly into the attached states to see if your host is in the list as expected.

Another valuable tool is the live log viewer, in order to use it, make sure to provide your rule with an easy to read description and enable the "log" option.

If you are using source routing (policy based routing), debugging can sometimes get a bit more complicated. Since the normal system routing table may not apply, it helps to know which flow the traffic actually followed. The packet capture is a useful tool in that case.

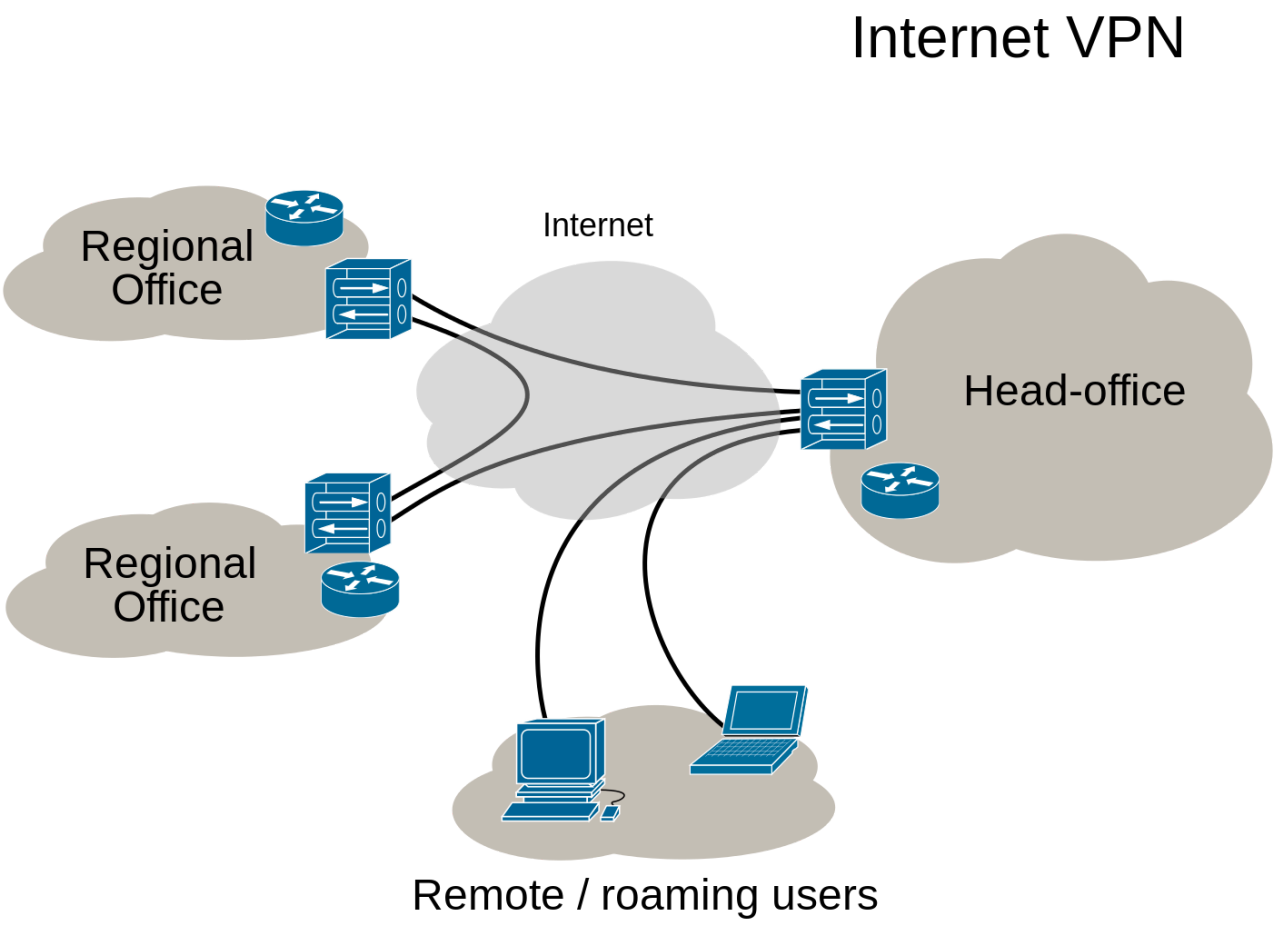
Common issues in this area include return traffic using a different interface than the one it came into, since traffic follows the normal routing table on it's way out (reply-to issue), or traffic leaving the wrong interface due to overselection (matching internal traffic and forcing a gateway).

Inspecting used netmasks is also a good idea, intending to match a host but providing a subnet is a mistake easily made (e.g. `192.168.1.1/32` vs `192.168.1.1/24` is in reality all of `192.168.1.x`).

Last but not least, remember rules are matched in order and the default (inbound) policy is `block` if nothing else is specified, since we match traffic on `inbound`, make sure to add rules where traffic originates from (e.g. `lan` for traffic leaving your network, the return should normally be allowed by state).

Virtual Private Networking

A virtual private network secures public network connections and in doing so it extends the private network into the public network such as internet. With a VPN you can create large secure networks that can act as one private network.



(picture from wikipedia)

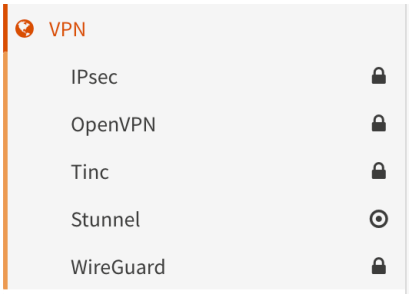
Companies use this technology for connecting branch offices and remote users (road warriors).

OPNsense supports VPN connections for branch offices as well as remote users.

Creating a single secured private network with multiple branch offices connecting to a single site can easily be setup from within the graphical user interface. For remote users, certificates can be created and revoked and a simple to use export utility makes

the client configuration a breeze.

OPNsense offers a wide range of VPN technologies ranging from modern SSL VPNs to well known IPsec as well as WireGuard and Zerotier via the use of plugins.



IPsec

Since IPsec is used in many different scenario's and sometimes has the tendency to be a bit complicated, we will describe different usecases and provide some examples in this chapter.

General context

The IPsec module incorporates different functions, which are grouped into various menu items. IPsec configurations are managed using Connections and the `swanctl.conf` format.

One of the main goals for the long run is to better align the gui components so they reflect the reality underneath, as we use StrongSwan, our aim is to follow their terminology more closely than we previously did.

The following functions are available in the menu:

- Connections
  - Configuration tool offering access to the connections and pools sections of the `swanctl` configuration
- Pre-Shared Keys
  - Define secrets to be used for local authentication.
- Key Pairs
  - For public key authentication collect public and private keys.
- Mobile & Advanced Settings
  - Define passthrough networks (to exclude from kernel traps), logging options and some generic options

- Status Overview
  - Shows tunnel status
- Lease Status
  - For mobile clients, show address leases for various pools configured
- Security Association Database
  - Shows security associations, the fundamental concept of IPsec describing a relationship between two or more entities
- Security Policy Database
  - Installed security policies describing which traffic is allowed to pass a tunnel
- Virtual Tunnel Interfaces
  - Edit or create new `if_ipsec(4)` interfaces
- Log File
  - Inspect log entries related to IPsec

Migrating from tunnels to connections

Having used the tunnel settings from the early OPNsense days, some terminology might be a bit confusing when moving into the new options offered. This paragraph aims to explain some of the common terms from the tunnel section and their new place in the connections. For a full list of changes, the upstream migration documentation is an interesting read as well.

- Phase 1 - The general connection settings, like local/remote addresses and general protocol settings. Choices in authentication to use are also part of this, they may involve multiple rounds.
- Phase 2 - Nowadays StrongSwan calls these **children**, as these define the `CHILD_SA` subsections in play. This is where you can define the networks on both ends. When multiple segments are being added into the same child, these are being treated as one policy where all of them are able to communicate to each other.
- Phase 1 / Tunnel Isolation - This option made sure every network defined in phase 2 would be treated as a child of it's own (e.g. two phase 2's would turn into two children)
- Phase 2 / Manual SPD entries - Manual SPD entries, this has been replaced with it's own menu option (Security Policy Database) offering more flexibility and visibility.

Note

Using DNS for endpoints is possible, but will work a bit different than previously as in most cases the firewall tried to resolve the names and didn't use the functionality provided by StrongSwan. It is however currently not possible to use DNS entries for

VTI tunnels due to restrictions in *if_ipsec(4)* as these type of interfaces can't be changed dynamically in a reliable way.

Note

When migrating Pre-Shared Key type tunnels to connections, make sure to add an entry in the "Pre-Shared Keys" module as well. If both ends should use their own identifier, fill in both local and remote values. The legacy module requested this information in the phase 1 page and wrote the same information to the secrets.

Since OPNsense uses the new StrongSwan format also for legacy tunnels, it is rather easy to convert a tunnel manually when downloading the `swanctf.conf` file from the machine. You can download it via **VPN -> IPsec -> Advanced Settings -> swanctl.conf**. The format is almost identical to the connections gui available in OPNsense.

Let's take a look at the following `swanctl.conf` that contains a legacy `con1` tunnel:

```
# This file is automatically generated. Do not edit
connections {
    con1 {
        unique = replace
        aggressive = no
        version = 2
        mobike = yes
        local_addrs = 203.0.113.1
        local-0 {
            id = 203.0.113.1
            auth = psk
        }
        remote-0 {
            id = 198.51.100.1
            auth = psk
        }
        encap = no
        remote_addrs = 198.51.100.1
        proposals = aes256-sha256-modp2048
        children {
            con1 {
                start_action = start
                policies = yes
                mode = tunnel
                sha256_96 = no
                local_ts = 192.168.1.0/24
                remote_ts = 172.16.100.0/24
                reqid = 1
                esp_proposals = aes256-sha256-modp2048
            }
        }
    }
}
pools {
}
secrets {
    ike-p1-0 {
        id-0 = 198.51.100.1
        secret = 0saGVsbG93b3JsZA==
    }
}
```

To convert this configuration:

- Go to **VPN -> IPsec -> Connections**
- Press **+** to add a new Tunnel and enable the *advanced mode*

Connections | Local Authentication | Remote Authentication | Children

Option	Value	swanctl.conf
Proposals	aes256-sha256-modp2048 (remove default)	proposals = aes256-sh
Unique	Replace	unique = replace
Version	IKEv2	version = 2
Local addresses	203.0.113.1	local_addrs = 203.0.113
Remote addresses	198.51.100.1	remote_addrs = 198.51
Description	con1-phase1	

Press *Save*, then move to *Local Authentication* and press **+**

Now we must configure a *Pre-Shared Key* that matches the *Local Authentication* and *Remote Authentication*:

- Go to **VPN -> IPsec -> Pre-Shared Keys**
- Press **+** to add a new *Pre-Shared Key*

Pre-Shared Key

Option	Value	swanctl.conf
Local Identifier	203.0.113.1 (required now)	
Remote Identifier	198.51.100.1	id-0 = 198.51.100.1
Pre-Shared Key	helloworld	secret = 0saGVsbG93b3JsZA==
Type	PSK	auth = psk
Description	con1-phase1	

Note

Gather the unhashed secret from **VPN -> IPsec -> Tunnel Settings**. Generally it is good practice to document these secrets in a password manager.

After the initial configuration, go to **VPN -> IPsec -> Tunnel Settings** and deactivate the *con1* tunnel by unchecking *Enabled* in *Phase1* and *Phase2*.

Go back to **VPN -> IPsec -> Connections** and ensure the new tunnel is *Enabled*. Press *Apply* and the new configuration will be active.

Combining legacy tunnels and connections

It is possible to combine tunnels and connections, but there are some constraints. As our legacy tunnels force a `reqid` for each configured child (phase 2), there is a risk the automated numbering from the new connection children overlaps. To prevent these overlaps, its required to set an unused `reqid` in the connection child.

Security policies and routing

In order to pass traffic over an IPsec tunnel, we need a policy matching the traffic. By default when adding a phase 2 (or child) policy a "kernel route" is installed as well, which traps traffic before normal routing takes place.

Note

Without a policy in place for the tunnel, traffic won't be accepted, in case a policy with a kernel route overlaps a local or locally routed network the traffic will not be received by the host in question.

Tip

When matching overlapping networks in a policy (VTI or overlapping networks), make sure to exclude your own network segments in the `Passthrough networks` option in **VPN -> IPsec -> Advanced Settings** to prevent traffic being blackholed.

Firewall rules

Connections do not add WAN rules automatically, so these have to be specified manually in order to connect to IPsec on this host.

The relevant protocols and ports for IPsec are the following:

- Protocol: ESP (https://en.wikipedia.org/wiki/IPsec#Encapsulating_Security_Payload)
- Port: 500/UDP (https://en.wikipedia.org/wiki/Internet_Security_Association_and_Key_Management_Protocol)
- Port: 4500/UDP (https://en.wikipedia.org/wiki/NAT_traversal#IPsec)

Note

One of the main reasons we are not offering automatic rules is that their either more open than expected (allow IPsec from anywhere) or too closed as the rule engine will "guess" the remote endpoint (in case of a fqdn).

The default behavior of our firewall is to block inbound traffic, which also means traffic using the tunnel should be allowed explicitly. In **Firewall ‣ Rules**, select the IPsec interface to manage the IPsec traffic policies.

Dead Peer Detection (DPD)

Dead Peer Detection (DPD) is a method of detecting a dead IKE peer as specified by RFC 3706.

When a peer is assumed dead, an action may be specified, such as closing the CHILD_SA or re-negotiate the CHILD_SA under a fresh IKE_SA.

Note

DPD is disabled by default, when using connections, make sure to specify a `DPD delay (s)` > 0 to enable the feature. Actions may be specified on its children.

When using IKEv1 a `dpdtimeout` can be specified to control when a peer is considered to be inaccessible. This setting has no effect on how IKEv2 handles retransmissions, in which case the general settings will be used as specified in the following document.

Warning

By default for IKEv2 the timeout on connections triggering a dpd action takes at least a couple of minutes, when quicker interaction is needed the `charon` retransmit timings should be changed which applies to all tunnels. These settings

can be changed via the Advanced settings, or when not yet supported on your version, a custom StrongSwan configuration.

Implementation schemes

When setting up IPsec VPNs there are two main types of scenario's with their own advantages and disadvantages.

Policy based

The first one is the standard policy based tunnel, which guards the security of the tunnel with policies and installs kernel traps to send traffic over the tunnel in case it matches these policies. For example a local network `192.168.1.0/24` sending traffic to a remote location responsible for `192.168.2.0/24`. The advantage of this scenario is the ease of setup, no routes are needed to be configured, when in this example `192.168.1.10` contacts `192.168.2.10` the packets are seamlessly forwarded over the tunnel to the remote location.

When local traffic doesn't match the policies in question due to the tunnel needing Network Address Translation, that's also possible as long as policies are manually added to the security policy database, this is also referred to as "NAT before IPsec".

Route based (VTI)

Route based, also known as VTI, tunnels are using a virtual interface known as `if_ipsec(4)`, which can be found under **VPN -> IPsec -> Virtual Tunnel Interfaces**. This links two ends of the communication for routing purposes after which normal routing applies. The "(Install) Policies" checkmark needs to be disabled in this case for the child definition. Usually the communication policy is set to match all traffic (either `0.0.0.0/0` for IPv4 or `::/0` for IPv6).

So the same example as the policy based option would need (static) routes for the destinations in question (`192.168.1.0/24` needs a route to `192.168.2.0/24` and vice versa), peering happens over a small network in another subnet (for example `10.0.0.1` <-> `10.0.0.2`) bound to the tunnel interface.

The advantage of this type of setup is one can use standard or advanced routing technologies to forward traffic around tunnels.

Important

Contrary to other physical or virtual interfaces, there are some important caveats to take into consideration when deciding to set up your network with VTI tunnels:

- By default, OPNsense can only filter on the `IPsec` group (`enc0`).
- VTI interfaces show up as regular interfaces in the GUI, but firewall rules specified on these interfaces will not apply by default. Instead, they must be specified on the `IPsec` group (`enc0`). The OS makes a hard distinction between filtering on the `IPsec` group and VTI interfaces. To flip this behavior, the following tunables must be specified:
  - `net.inet.ipsec.filtertunnel`, value `1`.
  - `net.inet6.ipsec6.filtertunnel`, value `1`.
  - `net.enc.in.ipsec_filter_mask`, value `0`.
  - `net.enc.out.ipsec_filter_mask`, value `0`.

  The downside is that policy-based tunnels (`enc0`) cannot be filtered anymore.
- If you require a combination of Policy-based and VTI-based tunnels in your setup, the recommended filtering approach is to specify all firewall rules on the automatically available `IPsec` group (`enc0`). Since all IPsec traffic passes through `enc0`, both policy-based and VTI-based traffic can be filtered here.
- NAT rules can be specified on VTI interfaces in pure VTI-based setups without issue. However, if you require NAT on the VTI-based traffic, but you are filtering on the `IPsec` group, you must enable the `Skip firewall rules` option in the Virtual Tunnel Interface configuration, so no (automatic) rules interfere with the traffic.

Warning

The most reliable VTI tunnel setups use static addresses on both ends of the tunnel as the `if_ipsec(4)` device matches both source and destination [#]. In recent versions of our product it is possible to auto-detect and reconfigure the tunnel on connect by keeping both local and remote addresses of the VTI empty in **VPN ▸ IPsec ▸ Virtual Tunnel Interfaces**.

Tip

Since VTI tunnels are bound to requestid's it is important CHILD_SA's are instantiated not more than once. To prevent duplicate children, configure *Unique* as Replace on the instance (advanced mode) and use a "trap" policy for the start action.

Road Warriors / Mobile users

IPsec may also be used to service remote workers connecting to OPNsense from various clients, such as Windows, MacOS, iOS and Android. The type of client usually determines the authentication scheme(s) being used.

Address pools for mobile clients can be configured per connection on the `Pools` tab in **VPN -> IPsec -> Connections**.

Note

If you are configuring Radius authentication, make sure to select the relevant Radius servers in **VPN -> IPsec -> Mobile & Advanced Settings** under Radius (eap-radius). This pool of servers will be shared across all connections.

The examples section contains various options available in OPNsense. Different examples from strongSwan are usually quite easy to implement as Connections follow the `swanctl.conf` format closely.

Examples

This paragraph offers examples for some commonly used implementation scenarios.

Connections (**VPN -> IPsec -> Connections**)

- IPsec - Policy based public key setup
- IPsec - Route based (VTI) PSK setup
- IPsec - NAT before IPsec
- IPsec - Roadwarriors IKEv2

Tip

The number of examples on our end is limited, but for inspiration it's often a good idea to walkthrough the examples provided by StrongSwan. Quite some swanctl.conf examples are easy to implement as we do follow the same terminology.

Note

Using Network Address Translation in policy based tunnels is different, due to the fact that the installed IPsec policy should accept the traffic in order to encapsulate it. The *IPSec BINAT* document will explain how to apply translations.

CARP considerations

When using IPsec in a high availability setup, it is important to understand the implications of the setup. Without assuming what the remote gateway looks like (which may be a single device or a high availability setup as well), the following considerations should be taken into account:

- For IKEv2, MOBIKE should be disabled. Due to the nature of CARP, a virtual IP in backup state will "disappear", which will trigger MOBIKE to try to re-establish the connection from a different available IP, thus overriding your "Local address" configuration. In a lot of cases this will be the primary IP of the WAN interface.
- In all cases (initiator, responder or both) the "Local Address" must be set to a CARP virtual IP.
- DPD must at least be configured on the peer to detect a non-responsive peer and reauthenticate the connection. DPD is usually the limiting factor in failover response time and is therefore the primary functionality to adjust to allow for faster failover. See the DPD section for more information and constraints.
- IPsec connections never failover seamlessly between primary and backup and always need a fresh IKE_SA. If quicker failover is required, dynamic routing with route-based tunnels is likely a better solution.

Tuning considerations

In normal routing/firewalling scenarios, most significant performance gains in terms of throughput are achieved by distributing flows across multiple CPU cores. In any physical or virtual setup, flow distribution is often determined by hardware RSS on a NIC, or virtual queues emulating this behavior. Such flows are processed by default on the receiving CPU (direct dispatch via netisr), making sure that traffic is processed in

parallel. The distribution mechanism (before any OS processing) making this possible often hashes the IP/TCP/UDP 5-tuple to determine on which CPU a flow should be handled. Proper distribution therefore requires proper uniqueness of packet headers.

When any tunneling takes place, as is the case for IPsec with ESP flows, many different traffic flows may be encapsulated inside a single IP header, reducing uniqueness of traffic and therefore the ability to distribute work. Unless NICs or virtualized queues can distribute work on ESP packets, which is rarely the case, OPNsense firewalls can often only receive such tunneled flows on a single CPU, because the outer IP header is often the same.

If you have a simple, single-tunnel, single-flow setup (two flows for in-out), you may benefit from using async crypto to dispatch cryptographic work to multiple processors. However, there is no guarantee this will speed up work as it is dependent on multiple factors.

In order to do so, add or change the following tunable in **System ▸ Settings ▸ Tunables**:

Note

`net.inet.ipsec.async_crypto` = **1**

As soon as an IPsec packet is received, decryption must be handled first before anything meaningful can be done with the actual data. This process can be parallelized with the command above, but because IPsec packets need to be processed in an explicit order, these have to be re-injected in the same one, often making performance gains negligible - especially when the machine is busy. The FreeBSD kernel queues decapsulated IPsec packets at all times, in contrast to regular ethernet frame handling (see technical note below). This also means that IPsec flows must adhere to the default single thread limitation after decryption. To fan out the traffic to multiple CPUs after decryption, you can increase the amount of threads bound to netisr:

Note

- `net.isr.maxthreads` = **-1** <– equal the number of cores in the machine

You can also be more specific and set only N amount cores to be used for ipsec traffic processing.

In very busy environments, this change likely has the most impact.

Lastly, but less practical, you can scale up IPsec by ensuring distribution earlier on in the chain by spreading out more flows across multiple IKE SAs (different source/destination selectors). In this scenario you must ensure multiple ESP flows coming from different source IP addresses when configuring site-to-site tunnels. You must combine this with the `net.isr.maxthreads` setting to keep the traffic distributed while it is routed through the system.

Technical background

Netisr can be thought of as the conceptual equivalent of Receive Packet Steering (RPS) in Linux. How packets are distributed throughout the system is dependent on the packet protocol.

Netisr allows for two basic modes of operation: direct dispatch or deferred (queued) dispatch. Netisr is always single-threaded by default in deferred dispatch mode to make sure the system adheres to the strong ordering requirements in protocols.

In direct dispatch mode, the network driver handling the packet receive buffer moves the packets up the stack. If the packet is to be routed out via a different interface, this entire process is handled in the same thread (including decryption for IPsec, unless async crypto is set), which doesn't exit until the packet is routed out. During this time, the receive buffer may fill with new packets. In this mode, the single-threaded limitation doesn't apply as the execution context remains on the receiving thread, of which there are often multiple via the data channels provided by the NICs/drivers.

In deferred dispatch mode, the first protocol allowing deferred processing will queue the packet, at which point the driver input routine will exit immediately, so it can process new packets faster. This decoupling allows netisr to control which flows are executed on which threads (and by extension, CPU).

When it comes to IPsec, packets are always queued via netisr after decryption. Because netisr is single-threaded in this mode, it's always queued to the same separate thread. The `maxthreads` option overcomes this limitation. Furthermore, the SPI value is used to calculate which thread the flow ends up on. This is true for both VTI- and policy-based configurations.

Miscellaneous variables

Path MTU Discovery

When trying to enforce path mtu discovery (PMTU), you need to make sure packets leave the network with the `DF` set. The kernel offers a tunable `net.inet.ipsec.dfbit` which offers 3 options, `0`, clear the bit on packets leaving the firewall (default), `1`, set the DF bit or `2` to copy the bit from the inner header.

Diagnostics

In order to keep track of the connected tunnels, you can use the **VPN -> IPsec -> Status Overview** to browse through the configured tunnels.

The **VPN -> IPsec -> Security Policy Database** is also practical to gain insights in the registered policies, when NAT is used, the additional SPD entries should be visible here as well.

When troubleshooting problems with your firewall, it is very likely you have to check the logs available on your system. In the UI of OPNsense, the log files are generally grouped with the settings of the component they belong to. The log files can be found in the “Log file” menu item.

Tip

When trying to debug various issues, the amount of log information gathered can be configured using the settings in **VPN -> IPsec -> Advanced Settings**.

Custom configurations

In some (rare) cases one might want to add custom configuration options not available in the user interface, for this reason we do support standard includes.

While the `swanctl.conf` and the legacy `ipsec.conf` configuration files are well suited to define IPsec-related configuration parameters, it is not useful for other strongSwan applications to read options from these files. To configure these other components, it is possible to manually append options to our default template, in which case files may be placed in the directory `/usr/local/etc/strongswan.opnsense.d/` using the file extension `.conf`

IPsec configurations are managed in swantcl.conf format (as of 23.1), merging your own additions is possible by placing files with a `.conf` extension in the directory `/usr/local/etc/swanctl/conf.d/`.

Warning

Files added to these directories will not be mainted by the user interface, if you're unsure if you need this, it's likely a good idea to skip adding files here as it might lead to errors difficult to debug.

Note

Prior to version 23.1 it was also possible to add secrets and ipsec configurations in `/usr/local/etc/ipsec.secrets.opnsense.d/` and `/usr/local/etc/ipsec.opnsense.d/`, with the switch to 23.1 these files are deprecated and should be manually migrated into swanctl.conf format.

OpenVPN (SSL VPN)

One of the main advantages of OpenVPN in comparison to IPsec is the ease of configuration, there are fewer settings involved and it's quite simple to export settings for clients.

General context

The OpenVPN module incorporates different functions to setup secured networks for roadwarriors and site to site connections. Since the start of our project we organized the openvpn menu section into servers and clients, which actually is a role for the same OpenVPN process. As our legacy system has some disadvantages which are difficult to fix in a migration, we have chosen to add a new component named `Instances` in version 23.7 which offers access to OpenVPN's configuration in a similar way as the upstream documentation describes it. This new component will eventually replace the existing client and server options in a future version of OPNsense, leaving enough time to migrate older setups.

Tip

When upgrading into a new major version of OPNsense, always make sure to read the release notes to check if your setup requires changes.

Note

OpenVPN on OPNsense can also be used to create a tunnel between two locations, similar to what IPsec offers. Generally the performance of IPsec is higher which usually makes this a less common choice. Mobile usage is really where OpenVPN excels, with various (multifactor) authentication options and a high flexibility in available network options.

The following functions are available in the menu (as of OPNsense 23.7):

- Instances
  - New instances tool offering access to server and client setups
- Servers
  - Legacy server configuration tool
- Clients
  - Legacy client configuration tool
- Client Specific Overrides
  - Set client specific configurations based on the client's X509 common name.
- Client Export
  - Export tool for client configurations, used for server type instances
- Connection Status
  - Show tunnel status
- Log File
  - Inspect log entries related to OpenVPN

Public Key Infrastructure (X.509)

OpenVPN is most commonly used in combination with a public key infrastructure, where we use a certificate authority which signs certificates for both server and clients (Also know as TLS Mode). More information about this topic is available in our Trust section.

Tip

As of version 24.1 OPNsense is able to use OCSP to validate client certificates when using the new Instances. Make sure `Use OCSP (when available)` is enabled in the trust section of the server instance and the CA used contains a proper `AuthorityInfoAccess` extension as described in our Trust section.

Firewall rules

To allow traffic to the tunnel on any interface, a firewall rule is needed to allow the tunnel being established. The default port for OpenVPN is `1194` using protocol `UDP`.

After communication has been established, it's time to allow traffic inside the tunnel. All OpenVPN interfaces defined in OPNsense are grouped as *OpenVPN*.

Tip

In order to use features as policy based routing or manual routes, you can assign the underlying devices and use them in a similar fashion as physical interfaces.

High availability [CARP]

When operating an OpenVPN server, there's not much needed to allow an active/passive setup for your environment other then using a virtual (CARP) address. As the server will stop receiving traffic when the virtual address doesn't it, the backup will eventually become out of service automatically.

In client mode, the OpenVPN instance needs to stop trying to reconnect when it's not in `MASTER` mode, the legacy client module shutsdown all instances directly attached to the interface. Our new instances module allows to select the `vhid` to track. In most cases an explicit bind isn't needed for a client, the default for a client is to use the `nobind` option.

Note

It's not possible to move between machines fully seamless as the client will have to reconnect in order to reach a valid state again.

Examples

This paragraph offers examples for some commonly used implementation scenarios.

Note

When using a site to site example with `SSL/TLS` instead of a shared key, make sure to configure "client specific overrides" as well to correctly bind the remote networks to the correct client.

Legacy (**VPN -> OpenVPN -> Client|Server**)

- Setup SSL VPN site to site tunnel
- Setup SSL VPN Road Warrior

New (**VPN -> OpenVPN -> Instances**)

- Setup SSL VPN site to site tunnel
- Setup SSL VPN Road Warrior

Client Specific Overrides

The mechanism of client overrides utilises OpenVPN `client-config-dir` option, which offer the ability to use specific client configurations based on the client's X509 common name.

It is possible to specify the contents of these configurations in the gui under **VPN -> OpenVPN -> Client Specific Overrides**. Apart from that, an authentication server (**System -> Access -> Servers**) can also provide client details in special cases when returning `Framed-IP-Address`, `Framed-IP-Netmask` and `Framed-Route` properties.

Note

Client specific overwrites will be written **after** authentication or client connect (depending on the type of setup). This in order for authentication services like RADIUS to be able to provision additional properties, such as tunnel and local networks.

A selection of the most relevant settings can be found in the table below.

Client Specific Overrides

Parameter	Purpose
Disabled	Set this option to disable this client-specific override without removing
Servers	Select the OpenVPN servers where this override applies to, leave emp

Parameter	Purpose
Common name	The client's X.509 common name, which is where this override matche
IPv[4|6] Tunnel Network	The tunnel network to use for this client per protocol family, when emp
IPv[4|6] Local Network	The networks that will be accessible from this particular client per pro
IPv[4|6] Remote Network	These are the networks that will be routed to this client specifically usi
Redirect Gateway	Force the clients default gateway to this tunnel

Note

When configuring tunnel networks, make sure they fit in the network defined on the server tunnel itself to allow the server to send data back to the client. For example in a `10.0.0.0/24` network you are able to define a client specific one like `10.0.0.100/30`.

To reduce the chances of a collision, also make sure to reserve enough space at the server as the address might already be assigned to a dynamic client otherwise.

Tip

When using topology "subnet" the netmask usually equals the one defined in the instance itself as the gateway being pushed to the client is the first address in the network and otherwise unreachable.

Troubleshooting common issues

The most common causes for non functional overwrites are caused by mismatches, in order to debug these, make sure to check the logs for messages like the following:

- `Locate overwrite for 'XXX' using server 'XXX' (vpnid: XXX)` << trying to find an overwrite (user authentication))
  - Usually followed by `user 'XXX' authenticated using 'XXX' XXX` showing username, authenticator used and optionally the overwrite type and filename.
- `client config created @ XXX` << file written on client connect (without user authentication)
- `unable to write client config for XXX, missing target filename` << no matching overwrite found (without user authentication)

By default overwrites are matched by certificate common name, when `Force CSO Login Matching` (legacy) or `Username as CN` (instances) are set the username will be used instead.

WireGuard

General context

WireGuard® is a simple yet fast and modern VPN solution, which in some cases is more convenient than IPsec or OpenVPN, certainly in terms of options you need to configure. In our experience IPsec is the fastest solution for site-to-site connections, but WireGuard is the simplest option to setup.

A WireGuard setup on our end exists of the following main components:

- Instances: in the WireGuard configuration these are called "interfaces" and they describe how the virtual `wgX` device on our end is configured in terms of addressing and cryptography.
- Peers: these are the clients that are allowed to connect to us, described by their optional remote address including the networks that are allowed to pass through the tunnel. Peers belong to one or more instances.

Instances

In order to configure an instance, we start by adding one in the gui and generate a keypair. The public key is usually required for the other end of the tunnel (peer). An unused port to listen on is required as well. The tunnel addresses are configured on the `wgX` device (which is always visible in **Interfaces ▸ Overview**).

By default, when "*Disable routes*" is not set, routes are created for each connected peer to the networks selected in "*Allowed IPs*", optionally only a single gateway route might be configured as well.

Note

When choosing tunnel addresses, make sure the network defined includes the addresses being used by the peers. For example when choosing `10.10.0.1/24` the `wgX` instance has this address configured and is able to accept a peer using `10.10.0.2/32`.

Tip

Remember to create a firewall rule to allow traffic to the configured port and inside the tunnel.

Peers

Peers define the hosts that we exchange information with, which might be a road-warrior type or a static destination, in which case you either provide or omit an "*Endpoint Address and Port*". At minimum you need the public key of the other party, optionally you may offer a pre-shared key as additional security measure. The "*Allowed IPs*" define the networks that are allowed to pass the tunnel.

Note

In most cases the "*Allowed IPs*" list contains the networks used on the remote host and the peer ip address (instance/tunnel address) configured on the other end.

Tip

When NAT and firewall traversal persistence is required, the :code:` Keepalive interval` can be used to exchange packets every defined interval ensuring states will not expire.

Peer generator

When creating login credentials for multiple clients, the peer generator offers a convenient way to create and configure them.

The generator lets you select the WireGuard instance the new peer should belong to and automatically derives suitable client addresses from the networks configured on that instance. It also stores generator defaults on the selected instance so they can be reused for future peers.

Warning

Since IP addresses are calculated before the peer is stored, creating multiple peers for the same instance concurrently can result in the same address being proposed more than once.

Each newly created peer receives a key pair. By default, only the public key is stored on the firewall.

Attention

Storing the peer private key on the firewall is optional and disabled by default.

The private key normally only belongs on the client device. Enabling private key storage makes it possible to reconstruct the generated client configuration later, but also means that the client's private key is retained in the firewall configuration.

Only enable this option when the convenience of being able to retrieve the client configuration again outweighs the additional exposure of storing the private key on the firewall.

After providing the relevant client information, the generated configuration can be copied from the `Config` field or imported using the displayed QR code.

When private key storage was enabled while creating the peer, the peer can later be opened again from the **Peers** page using the QR code action. This opens the peer generator in reference mode and reconstructs the client configuration and QR code from the stored peer and instance information.

Peers created without private key cannot have their original client configuration or QR code reconstructed later, because the required private key was never retained by the firewall.

The peer generator uses values stored on the selected instance as defaults for newly generated configurations. These include the client endpoint, DNS servers and allowed IPs and can be changed in the instance configuration.

Peer-specific values, such as the assigned tunnel address, preshared key and keepalive interval, remain part of the peer configuration.

High availability (using CARP)

When using WireGuard on active/passive high availability clusters, only one instance at a time is allowed to communicate to the other party. In OPNsense this can be reached by selecting a `vhid` to track as instance dependency {Depend on (CARP)}.

If an instance depends on a CARP vhid, it will query the current status and determine if the interface should be usable (when MASTER), the interface status (up/down) will be toggled accordingly.

Note

As the interface itself will not change, all of its addresses and routes remain when not being active. This ensures a relatively quick switch between roles.

Tip

Because the carp dependency is managed per instance, you are able to keep tunnels available selectively, for example to manage the machines remotely.

Diagnostics and debugging

In **VPN ‣ WireGuard ‣ Status** you can find the configured instances and peers including their last known handshake and the amount of data being exchanged. For Instances you are also able to see if the device underneath (`wgX`) is up or down, depending on the carp status described in the previous chapter.

Tip

Although WireGuard itself offers very limit logging, our setup process will make a note of errors and signal about certain events. When having issues configuring an instance or peer, always make sure to check the logs in **VPN ‣ WireGuard ‣ Log File** first.

Warning

When having issues exchanging packets between both ends of the tunnel, always make sure to check if the "*Allowed IPs*" in the peer configurations contain the proper networks. In case traffic is not allowed when traveling **in**, its dropped silently (a capture will not show it), roughly the same happens when traveling **out**, a capture will show it, but nothing will be send out.

Note

Runtime debugging from the console is possible using the `ifconfig` command, for more information see the upstream manual page

Examples

This paragraph offers examples for some commonly used implementation scenarios.

- WireGuard Site-to-Site Setup
- WireGuard Road Warrior Setup
- WireGuard AzireVPN Road Warrior Setup
- WireGuard MullvadVPN Road Warrior Setup
- WireGuard ProtonVPN Road Warrior Setup
- WireGuard Selective Routing to External VPN Endpoint

Plugin VPN options

Via plugins additional VPN technologies are offered, including:

- **OpenConnect** - SSL VPN client, initially build to connect to commercial vendor appliances like Cisco ASA or Juniper.
- **Stunnel** - Provides an easy to setup universal TLS/SSL tunneling service, often used to secure unencrypted protocols.
- **Tinc** - Automatic Full Mesh Routing
- **WireGuard** - Simple and fast VPN protocol working with public and private keys.
- **Zerotier** - seamlessly connect everything, requires account from zerotier.com, free for up to 100 devices.

- OpenConnect Setup
- Stunnel
- Zerotier Configuration

Access / User Management



The user manager of OPNsense allows for controlling access to the different part (pages) of the configurator as well as controlling access to particular services on a per user bases.

Authentication

OPNsense offers integration with external servers for services that require user authentication. These services include:

- IPsec
- OpenVPN
- Captive Portal
- Proxy

Integrated into OPNsense are the **Local User Database** and **Voucher Server**. The Voucher Server is intended to be used with the Captive portal.

External services currently supported are:

- LDAP (OpenLDAP, MS Active Directory, Novell eDirectory)
- Radius
- OpenID connect (BE only)

Authorization

Besides authenticating, user authorization to access parts of the configuration can also be setup with an external server, but in order to grant the appropriate privileges to the users they need to exist in OPNsense's local user manager. This way one can validate a user against its externally stored password and have a fine grained control over the configuration pages that user may access.

Note

When creating administration users with a configured shell, their naming scheme's are more strict (max 33 characters of a limited set). All other usernames are more flexible as no local shell accounts will be created.

Users, Groups & Privileges

When using the local user manager, either stand alone or on combination with an external authentication server one can define groups to combine a set of privileges for a specific user group. A user should be an individual, a group needs to be specific in such a way that anyone of that group can be granted the same access rights, called privileges.

Note

It's not always required to have users in your local database, when the remote server should merely answer the question if a user offers a valid user/password combination, most services can just push this question to the authenticating server. Constraints in some cases can be part of the authenticator as well. When the user should login to the firewall (for example to change settings or download a profile), a local user is always required as it serves as a linking pin to the ACL system.

Settings (Users) | Settings (Groups) | Settings (Privileges)

Option	Description
Defined By	Informational, defines source
uid	Internal number
Disabled	Option to disable to user for authentication

Option	Description
Username	This users name
Password	Configure a password for local login
Scrambled Password	Generate a random password for this login to avoid login
Full name	Informational, users full name
E-mail	Informational, email address this user has
Comments	Informational, custom notes
Preferred landing page	Preferred landing page after login or authentication failure
Language	Language to use in the GUI
Login shell	(admin users only) shell to use
Expiration date	Account valid until
Group membership	Groups this user is a member of
Privileges	Privileges to assign to this user, we prefer to use groups instead
OTP seed	One time password token
Authorized Keys	SSH Authorized Keys for this user (when a shell is configured)

Authentication services

Authentication services can be configured using the settings in **System ▸ Access ▸ Servers**. This includes both local accounts and remote authentication.

By default, OPNsense GUI login will use local accounts. This can be changed, however, by going to **System ▸ Settings ▸ Administration**, scrolling down to the “Authentication” group, and changing the ‘Server’ option.

Tip

The authentication part always uses a local “anchor” (user in the local database) to account for access rights, some remote servers are able to supply the required information by communicating the groups a user is member of after authentication.

Local account configuration

Settings for handling login via local accounts can be set by going to **System** ▸ **Access** ▸ **Servers**, then clicking the 'Edit' icon (a pencil) for 'Local Database'. Here, you can improve security of local user accounts by setting password length and complexity constraints.

An overview of available settings:

Policy	Enable password policy constraints.
Duration	[Policy] Password duration, optionally define how often the user should change his c
Length	[Policy] Minimum password length to require.
Complexity	[Policy] Enforce password complexity checks.
Compliance	[Policy] Require SHA-512 password hashing instead of Blowfish

Tip

When using One Time Passwords in combination with the local database, the same policy constraints apply as configured in "Local Database".

Note

Our default Blowfish is not part of AES and is not considered compliant with any Federal Information Processing Standards (FIPS). Although more secure than SHA-512, its use may not be permitted in some environments.

Configuration

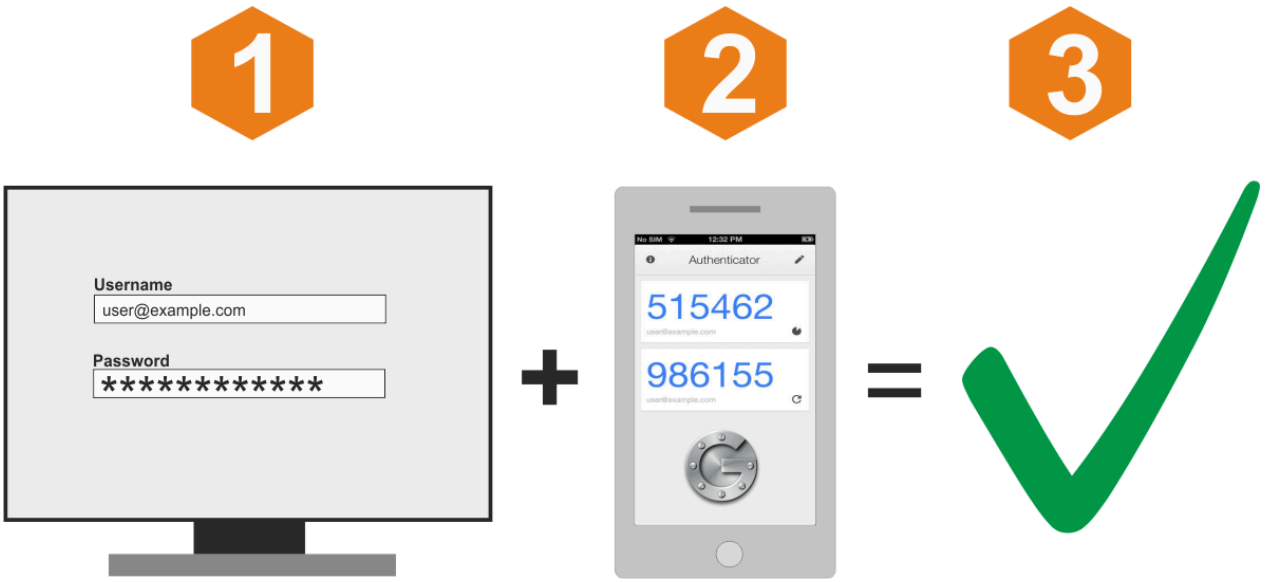
- Local Users & Groups
- Access / Servers / LDAP
- Access / Servers / Radius
- Two-factor authentication

Note

When using external authentication services (such as ldap), the firewall needs a route to the configured target machine in order to function properly. Specifically when the server is only reachable over a classic IPsec tunnel (non VTI), a static route might be needed to guide the packets into the right tunnel matching the policy. (for example when a policy for LAN exists, add a gateway pointing to our own address and a static routing pusing traffic to the other end).

A quick test if the routes are properly setup can always be performed using ping from the `default` source address.

Two-factor authentication



Two-factor authentication also known as 2FA or 2-Step Verification is an authentication method that requires two components, such as a pin/password + a token.

OPNsense (version >=16.1.14) offers support for Two-factor authentication throughout the entire system, with one exception being console/ssh access.

Supported services are:

- OPNsense Graphical User Interface
- Captive Portal
- Virtual Private Networking - OpenVPN & IPsec
- Caching Proxy

New authentication servers can be added via **System -> Access -> Servers**, which supports both local users and users synchronised via ldap.

Time-based One-time Password

TOTP is an algorithm that computes a one-time password from a shared secret key and the current time. OPNsense supports RFC 6238.

Google Authenticator

OPNsense fully supports the use of Google's Authenticator application. This application can generate tokens on Android, iOS and BlackBerry OS. The usage of this application is free and very simple to setup using OPNsense.

Other TOTP tokens

The 2FA feature can be used with any time based one-time password token, although it may be necessary to convert the tokens seed to the used format (base32).

Configuration & Setup

To setup see: Configure 2FA TOTP & Google Authenticator.